The OCEAN Handbook

Foundations-up reference for the OCEAN substrate-clustering language

Version 1.0 | LatentOcean Platform

Contents

Preface

> *After reading this preface you know who the book is for, what background it assumes, and how to read it.*

Who this book is for

The primary reader of this book has written code before. The book assumes familiarity with variables, functions, control flow, files, and the command line, at the level of a working data analyst or software engineer. It does not assume the reader has run a clustering pipeline, an embedding job, or an unsupervised learning analysis. Every substrate-clustering concept this book uses is introduced as it appears, with a worked example that the reader can run against a bundled toy corpus.

The secondary reader is a domain expert without a programming background: an auditor, a researcher, a journalist, a compliance officer, a scientist who has heard the words "deterministic" and "falsifiable" but has not seen a `.ocean` file. This reader is welcome. Each chapter has a "Wider system" sidebar that summarizes how that chapter's material fits the broader infrastructure story. Reading just those sidebars, in order, produces a coherent picture of why OCEAN exists and what it is for. The runnable snippets in between can be skipped without losing the thread.

What this book is not

This book is not the formal reference. The normative reference for the OCEAN language is at `docs/OCEAN_LANG.md` in the LatentOcean repository. That document is the language standard: every grammar rule, every type rule, every operator signature, in the terse and complete style of an ISO standard. When this book disagrees with the formal reference, the formal reference wins.

This book is not the primitive spec. The fingerprint primitive that underlies the proprietary embedding operator is documented at `docs/PRIMITIVE_SPEC.md`. Appendix C of this book is a short companion to that spec, but the spec itself is the contract.

This book is not the MCP server README. The OCEAN MCP server, which exposes the language as a set of tools for AI coding agents, is documented at `packages/ocean-mcp/README.md`. Chapter 12 of this book covers how to use it, but the README is the install guide.

What this book is: the one document a reader picks up to learn OCEAN from foundations, in the style of a "Programming Language" book rather than a standard or a manual page.

Three reading paths

Three documented paths through the book.

"Write OCEAN today." Chapters 1, 2, 4, 5, 6, 7, 8, 9, 10. Skim the others. About three hours.

"Understand the system." Preface, chapter 1, every "Wider system" sidebar in order, chapter 13, chapter 14. About one hour. This is the domain-expert path.

"The whole book." Cover to cover. About six hours read, two days work, if every exercise is attempted.

Conventions

Fenced code blocks marked ``` ``ocean are OCEAN source. Some of these are runnable in the web view of this handbook; clicking the **Run** button executes the snippet in a sandbox against one of the three bundled toy corpora. Snippets that depend on a local corpus or on premium operators are tagged static and do not have a Run button; the reader runs them by copying to a ```.ocean` file and using the compiler.

The first introduction of a term that has a glossary entry is set in italics, like *substrate* or *module*. Appendix D defines every italicized term.

The bundled toy corpora are described in Appendix B alongside the operator catalog.

The complete grammar is in Appendix A. The reference card on Appendix E fits on one printable page.

Where to start

The reader who is here for a quick answer: chapter 1 takes 20 minutes and ends with a six-line program that runs end-to-end on a real corpus.

The reader who is here for the full picture: the next paragraph is the first sentence of chapter 1. Start there.

What OCEAN Is

> *After this chapter you can answer the question "why does OCEAN exist?" in three sentences.*

Here are the three sentences, up front:

OCEAN exists to operationalize auditable systems at scale with deterministic mechanisms that ensure bit accountability. The language compiles to a directed acyclic graph of seven typed pipeline stages that runs identically across machines and seeds, so any number that lands in a downstream report can be re-derived bit-for-bit by the auditor who asks. The operator catalog is open-core; reference operators compose any pipeline end-to-end; proprietary operators carry the structural guarantees that make the audit contract commercially binding.

The rest of this chapter is the unpacking. Each of the next four sections expands one phrase from those three sentences.

Concepts in this chapter

- A domain-specific language vs a general-purpose library
- Determinism as a load-bearing contract, not a happy accident
- What *substrate-clustering* means and what it is good for
- The open-core split between reference and proprietary *operators*

A language for one job

OCEAN is a programming language with exactly one job: turn a corpus of records into a small number of stable, named groups called *modules*, and report how cleanly each group separates the labels you care about.

Here is a complete OCEAN program. Read it once. The vocabulary will be unfamiliar; that is fine. Each line corresponds to one of the seven *verbs* the language supports, and the next thirteen chapters explain each one in detail.

```ocean
static load tmp/corpus.ndjson
take 500 records balanced by archive
embed text into 128 dimensions using tf-idf
cluster for 16 rounds max 24 modules
energy = corpus mean
align modules using 50 nearest records
find dispersion of each label
save to data/result.json
```

> Six lines. The reader who is comfortable in Python or SQL is now thinking: "that is six function calls. Why is this a language and not a library?" It is a fair question. Here is the answer in three parts.
>
> **The pipeline shape is the same every time.** Every OCEAN program loads records, embeds them into a numerical space, optionally reduces them, clusters them into modules, aligns the modules back to the records, finds

````
the dispersion of each label across the modules, and saves an artifact.
That is the entire shape. A language can bake the shape in. A library cannot.
When the shape is baked in, the compiler can statically prove things about
the program that a library cannot prove about a sequence of function calls.

**The state between steps is opaque on purpose.** The output of `embed`
is a value of type `Z`. There is no way to ask what is inside it from
within an OCEAN program. The reader's first reaction is usually to want
to print the embeddings and look at them. That instinct is correct
for debugging and wrong for everything else. The opacity is what lets two
different embedders (TF-IDF projection vs MiniLM-L6) be hot-swapped under
the same `embed` verb without breaking anything downstream. The cost is
that OCEAN is not the tool to use for ad-hoc exploration. Use a notebook
for that.

**The compile target is a graph, not a script.** The six lines above
compile to a directed acyclic graph of operator invocations. The runner
executes that graph in topological order. Independent branches run in
parallel without the program author needing to say so. Cycles are
impossible by construction; `cluster` cannot consume its own output;
the type system forbids it. A library cannot enforce that. A language can.

## Determinism is the contract

The most surprising rule in OCEAN is this: for any program _P_, any seed
_S_, and any input file with a given content, executing _P_ at seed _S_
produces a byte-identical output file every time, on every machine.

```ocean static
seed 42
load tmp/corpus.ndjson take 500 records balanced by archive
embed text into 128 dimensions using tf-idf
cluster for 16 rounds max 24 modules energy = corpus mean
align modules using 50 nearest records
find dispersion of each label
save to data/result.json
````

The same six-line pipeline with a `seed 42` declaration. Run it twice on the same machine: identical output, byte-for-byte. Run it on a different machine with the same OCEAN version: still identical. The SHA-256 hash of the artifact file is the same. If it ever is not, that is a bug in OCEAN, not a feature of the data.

Determinism in OCEAN rests on four pillars, each enforced by the compiler:

> **Inputs are content-addressed.** Every `load` line stamps the SHA-256 of the file it loads into the operator's signature. If the file content changes by a single byte, every downstream operator's signature changes, and the runner refuses to reuse cached intermediates.
>
> **Operators are pure functions of their inputs and the seed.** No wall-clock, no system entropy, no thread-of-the-day. Where an operator would otherwise need randomness (k-means initialization, for instance), it derives the randomness from `(input_signature, seed)` and from nothing else.
>
> **Sweeps materialize in a stable order.** A `sweep s from 42 to 45 do ... end` block always produces its branches in ascending order of `s`. Two readers running the same sweep collect their results in the same sequence.
````

Parallel does not cross state. A `parallel do ... end` block isolates its branches from each other. They cannot share variables or write to the same output path.

Why this matters: OCEAN is the substrate beneath commercial claims about data. When a vendor publishes "the structural dispersion of this corpus is 0.73, with a null-test z-score of 18.19σ," the reader needs to be able to verify that number. Re-run the pipeline at the same seed, hash the artifact, compare. If the hashes match, the claim survived verification. If they do not, the claim is retracted. This is the falsifiability discipline that makes OCEAN-shaped infrastructure commercially viable, and it is the reason determinism is a language-level contract rather than a best-effort property.

A note on words: *determinism* and *reproducibility* are not the same thing. A program is *deterministic* when it produces the same output on the same machine every time. A program is *reproducible* when its output is the same on two different machines, possibly belonging to two different people. OCEAN promises both. Most ML pipelines, by contrast, promise neither; they assume you will run them once, look at the output, and not look at it twice.

What "substrate-clustering" means

OCEAN is a *substrate-clustering* language. The phrase has two halves, both load-bearing.

Substrate is the layer of structure underneath whatever a record appears to be about. A scientific paper appears to be about whatever the abstract describes. The substrate of that paper is the small number of mathematical structures it draws on. A patent appears to be about a specific invention. The substrate of the patent is the template of prior art it composes. A network packet appears to be about a particular flow. The substrate of the packet is the protocol shape that classifies whether it is benign or attack-shaped. The substrate is what survives translation across surface differences, and it is what unsupervised structural analysis finds.

Clustering is the act of grouping records by substrate, not by surface. A clustering algorithm is told nothing about labels; it infers structure from the embedding alone. After the algorithm runs, the reader can ask: "do the clusters happen to line up with a label the data already had?" If yes, the clustering has found something structural about the labeled distinction. If no, the clustering has found something the labels do not capture; which is often the more interesting case.

OCEAN is built around the assumption that this two-step move (embed records into a substrate-shaped space, then cluster them without labels) is the right tool for a specific class of problems:

Resystemizing entire processes. Workflows that ingest records, classify them, and decide outcomes can be re-stated as OCEAN pipelines. The rebuilt process is auditable end-to-end: every step is a deterministic operator with a stamped input signature, every module is interpretable by construction, and every claim derived from the pipeline is falsifiable against the saved artifact. Resystemization replaces a black-box process with one whose every decision is traceable to a hash and a seed.

Anomaly detection. When most records are "normal" and a few are structural outliers, substrate-clustering surfaces the outliers without being told what normal looks like in advance. The anomaly score is itself a deterministic function of the input; the same record scored at the

same seed against the same population produces the same score across machines and across time.

Outlier-biased data approaches. When the rare records are the signal, not the noise, an OCEAN pipeline can sample, embed, and cluster the corpus in a way that gives the outliers disproportionate weight. The BTUT pre-reduction operator is the proprietary tool that implements this; the free-tier path approximates the same effect with stratified sampling.

OCEAN is not built around classification, prediction, or generation. For those, use a different tool. OCEAN is the tool to reach for when the question is "what structure is in this data that has not been explicitly told to anyone yet?"

Open-core: what is free, what is paid

OCEAN ships an *operator catalog*: a finite list of named operators, each implementing one of the seven verbs in one specific way.

The catalog is *open-core*. Reference operators are free, fully documented, and sufficient to compose any pipeline end-to-end:

- `load.ndjson`: load NDJSON records
- `embed.tfidf_jl`: TF-IDF with Johnson-Lindenstrauss random projection
- `embed.transformer.minilm_l6`: MiniLM-L6 sentence-transformer embedder
- `cluster.kmeans`: k-means clustering with deterministic initialization
- `align.module`: module-to-record alignment via k-nearest
- `find.dispersion_per_label`: label-vs-module dispersion
- `persist.json`: pretty-printed JSON artifact with a SHA-256 sidecar

Proprietary operators are the algorithms that make OCEAN commercially distinctive. They require a paid API key:

- `embed.content_fp48`: Bloom-style 48-bit structural fingerprint
- `reduce.btut`: BTUT structural-anomaly pre-reduction
- `cluster.tcd_recursive_loop`: the TCD clustering algorithm with monotone module-energy guarantees
- `align.dispersion`: dispersion-weighted alignment

A reader who has only the reference operators can still run every pipeline in this book end-to-end. The premium operators add structural sharpness, scale, and the proprietary guarantees that back the commercial commitments in `docs/PRIMITIVE_SPEC.md`. The grammar is identical either way; the only difference at call site is the variant name after `using`. Swapping a free embedder for the premium `content fingerprint` variant is a single-word change.

This boundary is the same shape Postgres has between its core engine and its commercial forks (EnterpriseDB, Citus), or that Stripe has between its public SDK and the pricing-tier-gated APIs. The reference half is enough to learn and to validate; the premium half is what gets contracted.

Wider system

The previous four chapters of this section name three commercial analogies that bear thinking about. **Postgres** is the analogy for extension-driven adoption: the core is small, the surface is huge, and substrate status was earned by being installable everywhere and speaking a vocabulary that other tools borrowed. **dbt** is the analogy for verb-shape: a small grammar (model, ref, source, test) covers the entire job, and the verbs themselves became how a generation of data engineers describe their work, regardless of whether they run dbt. **Stripe** is the analogy for the open-core split: a public SDK that is genuinely good, and a separate commercial surface that monetizes the proprietary parts without locking the public parts behind a key.

OCEAN is built to follow the same arc. The vocabulary in the six-line pipeline above (*load*, *embed*, *cluster*, *align*, *find*, *save*, *modules*, *dispersion*, *seed*) is meant to become the way readers describe substrate-shaped problems in their own work, whether or not they ever write a `.ocean` file. The substrate-status play is not "OCEAN becomes ubiquitous because a marketing team made it ubiquitous." It is "OCEAN becomes ubiquitous because thinking in OCEAN's shape is the clearest way to think about a class of problems readers were already struggling to articulate."

Exercises

Look at the six-line snippet at the top of this chapter. One of those lines names an operator that is free-tier in the catalog. A different line names a verb that has both a free-tier and a premium variant. Identify both lines and the operators they name. (Hint: the catalog list above the "Wider system" section is the answer key.)

In your own words, write the difference between *determinism* and *reproducibility* as this chapter uses them. Two sentences is plenty. When done, check the glossary in Appendix D and compare.

What's next

Chapter 2 takes the six-line pipeline above, runs it end-to-end against a 50-record toy corpus, and reads the resulting JSON artifact line by line.

Your First Pipeline

> *After this chapter you have run a complete OCEAN pipeline end-to-end and read its output artifact.*

Concepts in this chapter

- The shape of an NDJSON *corpus*
- The six-verb canonical *pipeline* in execution form
- The structure of the JSON *artifact* a pipeline produces
- How the artifact's SHA-256 sidecar makes the run reproducible

The toy_tna_50 corpus

This book ships three small *corpora* for hands-on use. The first is `toy_tna_50.ndjson`, a 50-record sample drawn from a fictional national archive: every record names a piece of historical computing equipment and which archive collection it belongs to. The format is *NDJSON*: one JSON object per line, no enclosing array, no commas between records.

Here are the first three records of the corpus:

```
{"archive":"bombe","id":"tna-0000","primary_category":"structural","text":"machine catalogue entry
{"archive":"tunny","id":"tna-0001","primary_category":"mechanical","text":"machine catalogue entry
{"archive":"bombe","id":"tna-0002","primary_category":"electrical","text":"machine catalogue entry
```

Each record has four fields. The `id` is the stable record key. The `archive` field is a coarse-grained label, splitting records into two collections (`bombe` and `tunny`). The `primary_category` field is a finer-grained label, splitting records into three categories (`mechanical`, `electrical`, `structural`). The `text` field is what the pipeline will embed.

The toy corpus is small enough to run in under five seconds on a laptop. The conclusions a pipeline draws from 50 records are not statistically meaningful; the corpus is a teaching tool, not a benchmark. Chapter 9 covers the difference between a teaching corpus and a benchmark corpus.

Six lines that run

Here is a complete OCEAN program. It loads the toy corpus, embeds the text into a 64-dimensional space, clusters the embeddings into modules, aligns each module back to its closest records, finds the dispersion of the `archive` label across modules, and saves the result.

```ocean
run corpus=toy_tna_50 seed 42
load _toy_corpora/toy_tna_50.ndjson
take 50 records balanced by archive
embed text into 64 dimensions using tf-idf
cluster for 8 rounds max 6 modules
energy = corpus mean
align modules using 5 nearest records
find dispersion of each label
save to /tmp/first_pipeline.json
```

```
Run this snippet by clicking the **Run** button in the web view of this
handbook, or by saving it to `first_pipeline.ocean` and invoking the
compiler from a checkout of the repository:
```

python -m scripts.run_universal_pipeline --config first_pipeline.ocean

```
Either way, the program finishes in a few seconds and writes its
artifact to `/tmp/first_pipeline.json`. A second file is written
alongside it (`/tmp/first_pipeline.json.sha256`) holding the hex
digest of the artifact's bytes.

## Reading the output artifact

The artifact is a single JSON object. Here is the top-level shape,
truncated for the page:

```json
{
 "pipeline": {
   "seed": 42,
   "source_sha256": "8f2c...",
   "operator_versions": {
     "load.ndjson": "1.0.0",
     "embed.tfidf_jl": "1.0.0",
     "cluster.kmeans": "1.0.0",
     "align.module": "1.0.0",
     "find.dispersion_per_label": "1.0.0",
     "persist.json": "1.0.0"
   }
 },
 "modules": [
   {"id": 0, "size": 9, "centroid_hash": "a1b3...", "narrative": null},
   {"id": 1, "size": 8, "centroid_hash": "c742...", "narrative": null}
 ],
 "alignment": {
   "module_to_records": {
     "0": ["tna-0007", "tna-0019", "tna-0023"],
     "1": ["tna-0001", "tna-0012"]
   }
 },
 "dispersion": {
   "by_label": {
     "archive": {
       "bombe": 0.71,
       "tunny": 0.69
     }
   }
 }
}
```

The top-level object has four sections. The `pipeline` section is the provenance block: the seed used, the SHA-256 of the input file, and the version stamp of every operator that ran. Without these, the artifact is not reproducible. With them, any reader can re-run the pipeline at the same seed against the same source file and compare SHA-256 hashes.
```

The `modules` section lists the modules the clustering algorithm produced. Each module has an id, a size (how many records are closer to this module than any other), a centroid_hash, and a slot for an optional `narrate` annotation (Chapter 12 covers narration).

The `alignment` section maps each module back to its closest records. The default is the top five records per module; the `using K nearest records` knob in the program adjusts this.

The `dispersion` section is the headline finding. The score `0.71` on `bombe` means: across the six modules, the `bombe` archive is fairly well concentrated in a small number of modules rather than spread evenly across all of them. A score of `1.0` would mean a single module captures the entire archive; a score of `0.0` would mean every module contains exactly the same proportion of `bombe` records. The numerical reading of dispersion is covered in detail in Chapter 8.

What the six lines mean

Re-read the snippet with the artifact in front of you. Each line in the program corresponds to a section of the artifact.

Program line	Artifact section that records its result
`seed 42`	`pipeline.seed`
`load _toy_corpora/...`	`pipeline.source_sha256`
`embed text into 64 dimensions ...`	(intermediate, not in artifact)
`cluster for 8 rounds ...`	`modules`
`align modules using 5 ...`	`alignment`
`find dispersion of each label ...`	`dispersion`
`save to /tmp/...`	(writes the file itself)

The embedding step does not show up in the artifact. The 64-dimensional vectors are intermediate state. The clustering step records its result because clusters are durable; the embeddings are scaffolding.

This is a recurring shape in OCEAN: intermediate stages produce opaque in-memory values, and only the terminal stages produce durable named outputs that show up in the artifact. The hidden intermediates are what give the language room to swap operator implementations without changing what is recorded.

Wider system

Compare this six-line program to a typical Jupyter notebook that does the same analysis. The notebook would be 80 to 150 lines: imports, NDJSON reading, vectorizer setup, k-means parameter sweeps, alignment code, dispersion math, JSON serialization. The notebook would also be *unreproducible by*

default. Two readers running the same notebook get different random initializations, different vectorizer vocabularies, different module ids. Comparing across two notebook runs requires careful bookkeeping.

OCEAN's six lines collapse to one declarative DAG with a stamped seed and stamped operator versions. Comparing across two runs is two `sha256sum` invocations. That collapse, repeated across an organization with hundreds of pipelines, is the operational case for OCEAN: a notebook-shaped process becomes auditable not by adding checklists but by being a different shape of artifact.

Exercises

Change `dimensions` from 64 to 128 and re-run. Does the dispersion on `bombe` go up, go down, or stay roughly the same? Why might that be?

Change `max 6 modules` to `max 12 modules` and re-run. Each module now contains fewer records on average. How does the `module_to_records` alignment section in the artifact change?

Run the pipeline twice with the same seed and compute the SHA-256 of the artifact each time. The hashes should match. Now change `seed 42` to `seed 43` and re-run. Do the hashes still match? What does this tell you about the role of the seed?

What's next

Chapter 3 zooms in on the lexical level: which characters in an OCEAN program count as a token, an identifier, a literal, a keyword, or a comment.

Source Files and Tokens

> *After this chapter you can read any OCEAN source file at the lexical level: every comment, identifier, literal, keyword, verb, and operator.*

Concepts in this chapter

- UTF-8 source files with statement-significant newlines
- Comments, identifiers, and literal classes
- The split between reserved words and the verb namespace
- The full operator set

Source encoding and line endings

OCEAN source files are UTF-8 encoded. The conventional file extension is `.ocean`. Line endings may be either LF or CRLF; the lexer normalizes CRLF to LF on parse. Trailing whitespace on a line is ignored. Trailing newline at end of file is permitted but not required.

Newlines are *statement-significant*. A statement is terminated by a newline when the lexer is not inside a parenthesized expression or a brace block. The following two programs are equivalent:

```ocean static
load tmp/corpus.ndjson take 500 records embed text into 128 dimensions
```

```ocean static
load tmp/corpus.ndjson take 500 records ; embed text into 128 dimensions
```

OCEAN does not actually accept the `;` separator. The second program above is illegal. The point is that statement boundaries come from newlines, not from punctuation. There is no `;` and no `\` line continuation. A statement that wants to span multiple lines must do so inside parentheses or a brace block.

Comments

A `#` character starts a single-line comment. The rest of the line is ignored.

```ocean static
```

This program is documented in handbook chapter 11.

seed 42 load tmp/corpus.ndjson # in-line comment after a statement

```
Block comments (`/* ... */`) are not supported. The reasoning is the
same as in Python: a language with only one way to write a comment
makes diffs cleaner and machine-readable.

## Identifiers
```

ident ::= [a-z_] [a-z0-9_]*

```
Identifiers are all lowercase, with underscores. They start with a
letter or underscore and continue with letters, digits, or
underscores. Identifiers are case-sensitive in principle (the lexer
only recognizes lowercase, so `Records` is not a valid identifier in
the user namespace; it is a type name in the language namespace,
covered in Chapter 4).

Identifiers that start with a single underscore (`_corpus`,
`_intermediate`) are conventionally treated as private or
internal-only. The compiler does not enforce this; it is a style
convention.

Reserved words and verbs are not legal identifiers in declaration
positions (`let`, `define`, `import-as`). They may appear elsewhere
as field names in literal data, but not as the name of a `let`
binding or a `define`d function.

## Literals

OCEAN has six literal classes: integer, float, string, path, boolean,
and interpolation.

**Integers** are sequences of digits, optionally negative, with
optional underscore digit-grouping for readability:

```ocean static
let n = 500
let big = 1_000_000
let negative = -1
```

Floats use the usual decimal-point notation, with optional scientific notation:

```ocean static let pi = 3.14159 let small = 1.5e-9 let neg = -0.5

```
Strings are enclosed in double or single quotes. Both forms accept
the same content. The escape sequences supported are: `\"`, `\'`,
`\\`, `\n`, `\t`, `\r`.
```

```ocean static
let title = "TNA Hardware Catalogue"
let alt = 'single-quoted strings are also fine'
let with_escape = "this is line one\nthis is line two"
```

Paths are a special literal class for filesystem paths. A path literal is a sequence of word characters, slashes, dots, and dashes, ending in a known extension (`.ndjson`, `.csv`, `.tsv`, `.json`, `.yaml`, `.yml`, `.txt`, `.ocean`). Path literals do not need quotes.

```ocean static load tmp/corpus.ndjson save to data/result.json import "presets/atlas.ocean" as atlas

```
The third form quotes the path because the `import` syntax requires a
string literal. The first two are bare path literals.

Booleans are the lowercase words `true` and `false`:

```ocean static
let verbose = true
let stop_early = false
```

Interpolations are the string templating form, written `${name}`, used inside paths and strings to embed a binding value:

```ocean static sweep s from 42 to 45 do save to data/validation/seed_${s}.json end

```
The braces are required. `${s}` is the only legal form; `$s` is not.
The expression inside the braces is restricted to a single
identifier; full expressions are not supported.

Reserved words and verbs

OCEAN has a small reserved-word set. These names cannot be used as
identifiers in declaration positions:
```

require seed let in as on from to into using with by of do end sweep compare against parallel import step take balanced field is for rounds round max modules module energy crystallize every nearest records record dispersion each label fine anchored dimensions dimension loop recursive tcd tf-idf tfidf content fingerprint one-hot numeric mean corpus normal text define return if then else elif true false not and or

```
The boolean operators are spelled out as words (`and`, `or`, `not`)
rather than as symbols (`&&`, `||`, `!`). This is a deliberate choice;
SQL and Python use the same convention, and the resulting code is
more readable to non-programmers.

Eight names live in the _verb_ namespace, separate from the reserved
words:
```

load embed reduce cluster align find narrate save
```

```
A verb introduces a statement. Verbs and reserved words do not overlap;
no name is both. The list is closed by design, on the principle that
adding a new top-level verb would change the shape of every OCEAN
program.

## Operators

The full set of operators recognized by the lexer:
```

= assignment / named-arg separator , arg separator (optional, newline serves the same purpose) { block start } block end (expression group) expression group == equality != inequality < less-than

```
  greater-than
```

*<= less-or-equal = greater-or-equal + addition - subtraction / unary minus * multiplication / division*

```
OCEAN deliberately omits a number of operators that other languages
have: bitwise operators (`&`, `|`, `^`, `~`, `<<`, `>>`), modulo
(`%`), increment and decrement (`++`, `--`), the conditional ternary
(`?:`), and most of the C-derived family. Substrate-clustering
pipelines do not need them.

## Wider system

The lexical design follows from one premise: OCEAN programs are read
more often than they are written, and they are read by people whose
day job is not programming. A scientific data steward, a compliance
officer, a domain expert reviewing a vendor's pipeline. For those
readers, every reserved word with an English spelling (`and` instead
of `&&`, `using` instead of a flag) is one less symbol to look up.
The smaller operator set means the eye does not have to parse `<<=`
or `~`. The path literal that does not need quotes (`load
tmp/corpus.ndjson`) reads like a shell command, which is the
familiar shape for many of those readers.

This is the same design instinct that made SQL legible to non-programmers
in 1974 and that made dbt's templated SQL legible to analytics
engineers in 2018. Lexical accessibility is part of the substrate-status
play; vocabulary capture only happens if reading the language is easy.

## Exercises

1. Write the shortest legal OCEAN program. (Hint: zero statements is
   allowed.)

2. Find the lexical error in this snippet, in your head, without
   running the compiler:

```ocean static
load tmp/corpus.ndjson && embed text into 128 dimensions
```

3. Which of the following are legal identifiers? `Records`, `corpus_size`, `_intermediate`, `123records`, `take`, `my-var`.
```

What's next

Chapter 4 leaves the lexical level and steps up to the type system: the seven pipeline types, what produces each, and what consumes each.

The Pipeline Types

> *After this chapter you can name the seven pipeline types, the verb that produces each, and the verb that consumes each.*

Concepts in this chapter

- The seven pipeline types and what each holds
- Why static typing exists in a declarative DSL
- The producer-consumer table
- One subtype relation (`Aligned <: Modules`)
- How to read an OCEAN type error

Why static types in a pipeline language

A type system for a small DSL like OCEAN seems heavy at first. SQL, the canonical small DSL, exposes types only at column boundaries (`int`, `varchar(N)`, `timestamp`). The query as a whole is not typed; rows flow through it and the result is just a set of rows.

OCEAN does something stronger: every statement has a type, every binding has a type, every verb is a typed function from input types to an output type. The type-checker runs after the parser and before the compiler hands off to the runner. A program that does not type-check does not run.

The reasoning is the reasoning for static typing in any production codebase: the cost of a runtime type error in a multi-hour pipeline run is much higher than the cost of catching that error at compile time. A pipeline that loads a 50-million-row corpus, embeds it for forty minutes, and then crashes because `cluster` was applied to `Records` instead of `Z` has wasted forty minutes and a few dollars. The type-checker catches that crash in milliseconds, before any operator runs.

The seven pipeline types

OCEAN has exactly seven types in the pipeline namespace:

Type	Description
`Records`	A finite set of structured records loaded from a file.
`Z`	A latent embedding of a `Records` set; opaque internally.
`Modules`	A partition of a `Z` into a small number of named groups.

`Aligned`	Modules with their nearest-record assignments and optional narratives.
`Dispersion`	A finding: a normalized score per label per module.
`Artifact`	The persisted JSON output of `save`. Terminal type.
`Pipeline`	The type of a `compare`, `sweep`, or `define` body.

There are also four primitive types used inside expressions: `Number`, `String`, `Path`, `Bool`. The escape hatch `Any` exists for the rare `define` function whose parameter type cannot be inferred at the call site. The seven pipeline types above are the ones the verbs care about.

The naming follows the data: `Records` are records; `Z` is the standard math notation for a latent space; `Modules` are clusters under their substrate-clustering name; `Aligned` is the post-alignment shape; `Dispersion` is the finding; `Artifact` is the persisted file; `Pipeline` is the meta-type of any block that produces other typed values.

Operator signatures

Every verb has a fixed input-output signature:

```
load    : Path -> Records
embed   : Records -> Z
reduce  : (Z, Records) -> (Z, Records)
cluster : Z -> Modules
align   : (Modules, Records, Z) -> Aligned
find    : (Aligned, Records, Z) -> Dispersion
narrate : Aligned -> Aligned
save    : (Aligned | Dispersion | Modules | Records) -> Artifact
```

`reduce` takes the pair `(Z, Records)` and returns a new pair of the same shape, filtering both down to a smaller surviving subset (the BTUT pre-reduction operator).

`align` and `find` each take three inputs: the directly relevant type, plus the original `Records` and `Z` carried forward in the DAG. The two carried inputs are wired implicitly by the compiler; the program never names them.

`save` is the only verb whose input type is a union. It accepts the most natural terminal value from any branch of the DAG.

Subtyping: Aligned is a Modules

OCEAN has one subtype relation: `Aligned <: Modules`. An `Aligned` value is a `Modules` value with extra information attached (the per-module nearest-record list and any narrative annotation). This means `narrate` and `find` can be skipped and the program still type-checks; `save` accepts an `Aligned` wherever a `Modules` is allowed.

```ocean static load tmp/corpus.ndjson take 500 records embed text into 128 dimensions cluster for 16 rounds max 24 modules using kmeans energy = corpus mean save to data/just_modules.json

```
This program saves a `Modules` value directly, without aligning or
finding dispersion. It type-checks because `save` accepts `Modules`.
The resulting artifact has the `modules` section but no `alignment`
or `dispersion` sections.

Reading a type error

When a verb is applied to the wrong input, the compiler emits a
diagnostic with a caret, a category, and a suggestion. Here is an
intentional type error and the diagnostic it produces:

```ocean static
load tmp/corpus.ndjson take 500 records as raw
cluster raw using tcd recursive loop
```

The compiler refuses to compile this program and prints:

```
ocean: error at file.ocean:2:1

 2 | cluster raw using tcd recursive loop
     ^^^^^^^

type error: cluster expects Z, got Records (from 'raw')

hint: pipe through embed first, e.g.:
       let z = embed text from raw into 128 dimensions
       cluster z using tcd recursive loop
```

The diagnostic carries four pieces of information: the file and position, the offending token, the actual-versus-expected types, and a typed suggestion. The suggestion is generated from the verb signature table and the upstream binding shape; it is not a generic "check your types" message.

Chapter 7 of this handbook covers `cluster` in detail and explains the `using tcd recursive loop` variant, which is a premium operator. For now, the only point is that the typechecker caught the error before the pipeline tried to load a 50,000-record file and cluster its `Records` directly.

Wider system

The seven-type pipeline namespace is deliberately fewer than would arise from a general data-flow language. Spark, in contrast, has at least three pipeline-shaped types (RDD, DataFrame, Dataset), each with its own operator set, and the conversions between them are a source of confusion for new Spark users. OCEAN avoids the choice by having a single linear progression: `Records` to `Z` to `Modules` to `Aligned` to `Dispersion` to `Artifact`. Every program is the same shape.

The substrate-status angle: the type names are part of OCEAN's vocabulary. When developers say "the dispersion of this label looks low" or "the modules came out badly," they are speaking OCEAN even if they are not running the language. That vocabulary capture is harder to achieve with a sprawling type system.

Exercises

1. Without running the compiler, predict the type error in the following program and write it out in the diagnostic format above:

```
ocean static load tmp/corpus.ndjson take 500 records align modules using 5 nearest records
```

   Name the verbs that accept `Dispersion` as an input. (Hint: only one.)

   The four types that `save` accepts as input are `Records`, `Modules`, `Aligned`, and `Dispersion`. Why not `Z`? What would it mean to save a `Z` value?

What's next

Chapter 5 starts the verb-by-verb tour with `load`, the only verb that takes a `Path` and produces a `Records`.

Load and Records

> *After this chapter you can load any NDJSON corpus, stratify the sample, and choose which fields hold the text and the label.*

Concepts in this chapter

- The NDJSON file shape
- `take N records` for size control
- `balanced by FIELD` for class-balanced sampling
- `text field is FIELD` and `label field is FIELD` for field selection
- What a `Records` value contains in memory

NDJSON: one JSON object per line

`load` accepts a single file format: *NDJSON*, also called "JSON Lines." One JSON object per line, no enclosing array, no separator characters between records.

```
{"id":"tna-0000","archive":"bombe","text":"..."}
{"id":"tna-0001","archive":"tunny","text":"..."}
{"id":"tna-0002","archive":"bombe","text":"..."}
```

NDJSON is the format because it streams cleanly (the loader does not need to read the whole file before processing any record), it is diff-friendly (a one-record change is a one-line change), and it preserves field order in every implementation. CSV is too lossy (every value is a string), Parquet is overkill for a teaching language, and JSON-array files force the loader to buffer the entire file.

A minimal load is one line:

```ocean static load tmp/corpus.ndjson

```
This loads every record in the file. With no sampling and no field
overrides, the loader picks the `text` field as the text body and
the `archive` field as the gold label, both by default.

Sampling: `take` and `balanced by`

For corpora larger than the laptop budget, `load` accepts a sampling
clause:

```ocean static
load tmp/corpus.ndjson take 500 records
```

The loader reads the entire file but emits only the first 500 records to the downstream verb. If the file has fewer than 500 records, the loader emits all of them and continues without warning.

For class-balanced sampling, add a `balanced by FIELD` clause:

```ocean static load tmp/corpus.ndjson take 500 records balanced by archive

> ```
> This samples 500 records spread evenly across the distinct values of
> the `archive` field, round-robin. If the file has two `archive`
> values (`bombe` and `tunny`), the loader emits 250 of each. If there
> are five values, the loader emits 100 of each. If a value has fewer
> records than the per-class budget, the loader takes all of that
> value and continues round-robin on the remaining values.
>
> The `balanced by` knob is the most underrated knob in the language.
> Without it, a corpus with a 95-5 imbalance produces clusters that
> faithfully reproduce the imbalance, and the dispersion finding looks
> roughly the same on every label. With it, the clusters have to find
> structure that distinguishes the smaller class from the larger one,
> which is often where the substrate signal lives.
>
> ## Pointing at the right fields
>
> The default `text` field is the field literally named `text`. The
> default label field is the field literally named `archive`. Both
> defaults can be overridden:
>
> ```ocean static
> load tmp/legal_cases.ndjson
>     text field is body
>     label field is jurisdiction
> ```

`text field is body` tells the loader that the `body` field of each record contains the text to embed. `label field is jurisdiction` tells the loader that the `jurisdiction` field is the gold label that `find dispersion of each label` will compute against.

If a record is missing the text field, the loader treats it as an empty string and includes the record. If a record is missing the label field, the loader assigns it the special label `_unlabeled` so the dispersion math has somewhere to put it.

There is also a `fine label field is FIELD` clause, used during alignment to map each module to a more granular label than the coarse one used by `find dispersion`. See chapter 8 for the alignment side of this knob.

What a `Records` value contains

A `Records` value is the in-memory representation of the loaded file. Conceptually it carries:

- The SHA-256 of the source file, for the determinism contract
- The records themselves (id, text, label, fine_label, fields)
- A canonical iteration order (the order records came out of the loader, which is deterministic given the file content and the sampling parameters)
```

Records is opaque from inside the OCEAN program. No verb other than `load` constructs a `Records` value, and no expression inside the language can ask "what is the text of record 5?" The records are black-box inputs to the downstream pipeline. The artifact saved by `save to PATH` does include selected record ids (in the alignment section, where the program asks `align modules using K nearest records`), but it never dumps the full record contents.

This opacity is deliberate. It prevents a class of bugs where two runs differ because the program author printed records in some nondeterministic order during development. It also prevents the artifact from accidentally exfiltrating sensitive text fields. The artifact records what the pipeline *decided*, not what it *read*.

Wider system

Compare OCEAN's `load` to a `COPY` in PostgreSQL or a source declaration in dbt. In all three cases, one short statement maps a file format into a logical, query-able shape. PostgreSQL's `COPY` needs a target table definition; dbt's source needs a YAML schema file; OCEAN's `load` needs neither. The schema is inferred from the first record at parse time, and the rest of the program references the field names by string.

The trade-off is that OCEAN is not the tool to use when the schema is unknown. If the records have wildly varying shape, or if half the records are missing the `text` field, the loader will not catch that. It will load the corpus, embed the empty strings, and produce a meaningless artifact. The fingerprinting primitive in `docs/PRIMITIVE_SPEC.md` has the same property: garbage in, garbage out, but the garbage will be deterministically the same garbage across machines.

Exercises

Write a `load` statement that takes 100 records from the file `_toy_corpora/toy_nslkdd_200.ndjson`, balanced across the field named `type`.

What happens if `take N` asks for more records than the corpus actually contains? Read the section "Sampling" again and write the answer in one sentence.

The toy_climate corpus has a `region` field and a `year` field. If a program wants to dispersion-test the four climate regions, what should `label field is` point at: `region` or `year`?

What's next

Chapter 6 covers `embed`: turning a `Records` value into a `Z` latent-space value, with three free-tier embedders and one premium variant.

Embed and Z

> *After this chapter you can choose between three free-tier embedders, name the premium structural variant, and pick a sensible target dimension.*

Concepts in this chapter

- What `embed` produces (the `Z` latent space)
- TF-IDF with Johnson-Lindenstrauss projection (free)
- The MiniLM-L6 sentence-transformer embedder (free)
- The one-hot numeric embedder (free, for non-text corpora)
- The premium `content fingerprint` variant (referenced, not run here)
- How to choose `into N dimensions`

What `embed` produces: the Z space

`embed` takes a `Records` value and produces a `Z` value. The `Z` value is a fixed-dimension numeric array, one row per record. The array dimensions are determined by the `into N dimensions` clause. The numerical contents of the array are opaque to the rest of the program. Nothing in OCEAN reads back the individual numbers; the downstream verbs operate on the array as a whole.

The minimal `embed` statement is:

```ocean static
embed text into 128 dimensions
```

> This reads the `text` field of each record (or whatever `text field is` set during `load`) and produces a 128-dimensional embedding using the default operator, TF-IDF with Johnson-Lindenstrauss projection.
>
> The embed verb has three free-tier variants and one premium variant.
>
> ## tf-idf (the default, free-tier)
>
> ```ocean static
> embed text into 128 dimensions using tf-idf
> ```

The free-tier default. The corpus is tokenized, term frequencies are computed, inverse document frequencies weight the rare terms, and a Johnson-Lindenstrauss random projection collapses the resulting high-dimensional sparse vector down to the target dimension.

TF-IDF is fast (under a second on 50,000 records on a laptop) and well-understood. The substrate it produces is *surface vocabulary*: two records that use the same words land near each other in the Z space. This is right for many tasks (topic clustering, exact-quote detection) and wrong for others (paraphrase matching, cross-language analysis).

Three optional knobs:

```ocean static embed text into 128 dimensions using tf-idf with min_df = 2 with max_df = 0.8 with max_features = 50000

`min_df` drops terms that appear in fewer than N documents.
`max_df` drops terms that appear in more than the given fraction of
documents (stopword-like terms). `max_features` caps the vocabulary
size.

## transformer minilm_l6 (free-tier, semantic)

```ocean static
embed text into 384 dimensions using transformer minilm_l6
```

A sentence-transformer embedder using the `all-MiniLM-L6-v2` model from sentence-transformers. The native output dimension is 384; ask for a different dimension and a linear projection is applied.

The MiniLM-L6 embedder is the right choice when the corpus is short text and the substrate of interest is semantic, not lexical. Two records that paraphrase each other will land near each other in the Z space, even if they share no words. The trade-off is speed: MiniLM is roughly an order of magnitude slower than TF-IDF, and it adds a ~90 MB model dependency to the runtime.

This variant is free-tier; the model weights are open-source.

one-hot numeric (free-tier, for non-text corpora)

```ocean static embed text into 64 dimensions using one-hot numeric

Despite the verb spelling `embed text`, this variant ignores the
text field. It reads the numeric and categorical fields of the
records and constructs a one-hot encoded representation, padded or
projected to the target dimension. This is the right embedder for
network-traffic logs, telemetry, or other tabular non-text corpora.

For the `toy_nslkdd_200.ndjson` corpus, which has fields like
`duration`, `protocol`, `service`, `src_bytes`, this variant
produces a meaningful Z that captures the relationships between
records based on their numeric and categorical attributes.

## The premium content fingerprint variant

The premium embedder is referenced by name but does not run in the
sandboxed handbook runner:

```ocean static
embed text into 48 dimensions using content fingerprint
```

The output dimension is always 48 (the width of the structural fingerprint primitive defined in `docs/PRIMITIVE_SPEC.md`). The fingerprint is a Bloom-style hash of the top-K most-distinctive terms of each record, with a rotation ensemble that yields 48 independent pseudorandom bits from a single SHA-256.

The substrate `content fingerprint` produces is *structural shape*, not surface vocabulary. Two records with completely different words but the same underlying template land near each other in the Z space. This is the right embedder for cross-corpus matching, template detection, and structural anomaly hunting.

Because the fingerprint is the proprietary primitive that backs the commercial commitment in `docs/PRIMITIVE_SPEC.md`, executing `content fingerprint` requires a paid API key. The grammar is identical either way; the runtime gate is the only difference.

Choosing into N dimensions

The target dimension is a knob the program author sets. The two considerations:

- **Statistical power.** Smaller dimensions concentrate signal but lose detail. 32 dimensions is the floor for any real corpus; below that, clusters get noisy.
- **Compute cost.** Larger dimensions cost more in every downstream operator: clustering quadratic in the dimension, alignment near-linear, dispersion negligible. 1024 is the practical ceiling for a laptop; 256 is comfortable.

Reasonable defaults by embedder:

Embedder	Sensible dimension range	Default in this book
`tf-idf`	64 to 256	128
`transformer minilm_l6`	384 (native) or down to 128	384
`one-hot numeric`	32 to 128	64
`content fingerprint`	48 (fixed)	48

When in doubt, run the same pipeline at two dimensions and use `compare` (chapter 10) to see whether the dispersion changes meaningfully.

Wider system

The split between TF-IDF and the content fingerprint is the open-core boundary made visible at the verb level. TF-IDF is well-understood public-domain mathematics. `content fingerprint` is the proprietary structural primitive. The verb is the same. The program author can prototype on TF-IDF, validate that the pipeline shape is right, and swap to `content fingerprint` for the commercial run by changing one phrase. That cost-of-switching matters: a customer who has already invested in OCEAN's vocabulary and pipeline shape can adopt the premium primitive without rewriting anything else.

This is the same shape Stripe uses for its test-mode and live-mode APIs. The API surface is identical; only the keys and the cost profile change. The substrate-status play depends on having that ergonomic continuity between free and paid.

Exercises

Pick the best free-tier embedder for each of the following corpora and justify in one sentence: - 10,000 news articles in English - 5,000 short tweets across 12 languages - 100,000 network traffic logs from a firewall - 500 historical letters whose authors are at issue

The toy_climate corpus has 100 records. Run an `embed` at 32 dimensions, then at 256 dimensions. Which artifact has a higher `dispersion.by_label.region` score? Why might that be?

The premium `content fingerprint` operator always produces 48 dimensions. If a program asks `into 64 dimensions using content fingerprint`, what should the compiler do? (Hint: look at the premium operator card in Appendix B.)

What's next

Chapter 7 takes a `Z` value and clusters it into `Modules`, with the free-tier `kmeans` operator and the premium `tcd recursive loop`.

Cluster and Modules

> *After this chapter you can run a clustering pass with sensible defaults and explain why the defaults are sensible.*

Concepts in this chapter

- What a `module` is and what fields it carries
- The free-tier `kmeans` operator
- The premium `tcd recursive loop` operator
- The energy function: `corpus mean` vs `normal anchored on LABEL`
- The loop parameters: `for N rounds`, `max M modules`, `crystallize every K`

What a module is

A `module` is a named, stable group of records that the clustering operator has decided belong together. After `cluster` runs, the program has a `Modules` value: a list of modules, each with an id, a centroid (the geometric center of the group's records in Z space), and a count of how many records the module contains.

```ocean static load tmp/corpus.ndjson take 500 records balanced by archive embed text into 128 dimensions cluster for 16 rounds max 24 modules using kmeans energy = corpus mean

```
This program produces between two and twenty-four modules, depending
on how many distinct clusters the algorithm finds. The `max M
modules` is an upper bound, not a target. The algorithm may produce
fewer than M modules if the data does not support that many.

A module is named by its id (an integer from 0). Modules in a
`Modules` value are sorted by id, and ids are assigned in a
deterministic order tied to the centroid hash. Two runs of the same
program at the same seed produce the same module ids for the same
records.

cluster.kmeans (the free-tier baseline)

```ocean static
cluster for 16 rounds max 24 modules using kmeans energy = corpus mean
```

The free-tier clustering operator is standard k-means with two non-standard properties:

> **Deterministic initialization.** The initial centroids are seeded from `(input_signature, seed)` rather than from system entropy. Two runs at the same seed start in the same place.

> **Stopping rule.** The algorithm stops when modules stabilize or when `for N rounds` is exhausted, whichever comes first. The default round count is 16; this is more than enough for most real corpora.

`kmeans` is fast (linear in records, quadratic in dimensions) and well-understood. It is the right choice for prototyping a pipeline and for any corpus where the modules are roughly spherical in Z space.

cluster.tcd_recursive_loop (the premium variant)

```ocean static cluster for 16 rounds max 24 modules using tcd recursive loop crystallize every 4 energy = corpus mean

```
The premium clustering operator is the TCD recursive loop, the
proprietary algorithm that gives OCEAN its substrate-clustering
guarantees. Three properties distinguish it from k-means:

1. **Monotone module energy.** Each loop round either lowers the
   total module energy (the sum of distances from each record to its
   module's centroid) or leaves it unchanged. The algorithm never
   oscillates.

2. **Bounded module count.** The `max M modules` is a hard cap, not
   a soft preference. The algorithm provably produces at most M
   modules. This matters for downstream operators that have to
   reason about per-module work.

3. **Crystallization.** Every K rounds, modules whose centroids
   have moved less than a threshold are _crystallized_: marked as
   stable and excluded from further centroid updates. This focuses
   the remaining rounds on the still-moving modules and lets the
   algorithm converge faster on long corpora.

The `tcd recursive loop` is a premium operator; executing it requires
a paid API key. The grammar above type-checks in the free-tier sandbox
but does not run. To run it locally, set `OCEAN_API_KEY` and use the
full compiler.

Energy functions: corpus mean and normal anchored

The `energy = ...` clause picks the optimization target for the
clustering operator. Two options:

```ocean static
cluster for 16 rounds max 24 modules energy = corpus mean
```

`corpus mean` (the default): minimize the sum of squared distances from each record to its module centroid, measured against the corpus mean. This is the standard k-means objective and is the right choice when no label is privileged.

```
ocean static cluster for 16 rounds max 24 modules energy = normal anchored on type
```

`normal anchored on LABEL`: pick the records whose value of LABEL is the literal string `normal` and use *their* centroid as the energy anchor. The algorithm then biases module construction so that the

"normal" records form one tight module, and the remaining records are partitioned according to how they differ from normal.

This is the right energy function for anomaly detection on a corpus where there is a well-defined normal class. The NSL-KDD intrusion detection corpus has a `type` field whose `normal` value names the benign network traffic; anchoring on that produces modules corresponding to attack patterns rather than to the geometry of the corpus as a whole.

Loop parameters

The clustering operator accepts three loop-shaping knobs:

`for N rounds` sets the maximum number of loop iterations. 16 is the sensible default for most corpora. For very large corpora (above 100k records), 32 or 64 may be needed. For very small corpora (the 50-record toy), 8 is plenty.

`max M modules` caps the number of modules the algorithm may produce. Choose this number by considering the expected coarseness of the substrate. If the corpus has roughly K underlying classes, set `max` to 2K or 3K; the over-provisioning gives the algorithm room to split classes that have internal structure.

`crystallize every K` (premium only) freezes converged modules every K rounds. Smaller K is more aggressive crystallization; K=1 freezes modules as soon as they converge, K=N never crystallizes anything. Use K=4 for most pipelines.

Wider system

Why the TCD recursive loop is the part of OCEAN that took the longest to design: it is the proprietary clustering algorithm whose guarantees (determinism, monotone energy, bounded module count) are what made the substrate provable. Without those guarantees, the language could not promise byte-identical artifacts; clustering algorithms with stochastic convergence would be a source of drift.

The substrate-status angle: a customer who prototypes on `kmeans` and swaps to `tcd recursive loop` for production is doing what the language was designed to enable. The verb is the same. The energy clause is the same. The loop parameters are the same. Only the variant phrase changes. This is exactly the ergonomic continuity that the open-core boundary requires.

Exercises

> Run `cluster for 8 rounds max 6 modules using kmeans energy = corpus mean` on the toy_tna_50 corpus. How many modules does the artifact's `modules` section list?
>
> The NSL-KDD corpus has a `type` field where `normal` names the benign records and `neptune`, `smurf`, `back` name attack types. Write a `cluster` statement that uses `normal anchored on type` as the energy function. (Hint: the `type` field is a string, the energy clause references it by name.)

What does `crystallize every K` mean? Explain in one sentence without using the word "crystallize."

What's next

Chapter 8 connects modules back to records via `align` and turns the result into a finding via `find dispersion of each label`.

Align and Find

> *After this chapter you can read a dispersion artifact and explain what it claims about a corpus.*

Concepts in this chapter

- Module-to-record alignment via k-nearest
- The `fine label field is` knob for finer-grained alignment
- The `find dispersion of each label` operation
- What dispersion measures (a single normalized score per label per module)
- Why dispersion is the ungameable falsifiability gauge

align: putting records back next to their modules

`cluster` produces a `Modules` value whose entries name the centroids of each cluster but do not list which records are nearest. `align` fills that gap by attaching, to each module, the k records whose Z positions are closest to that module's centroid.

```ocean static align modules using 5 nearest records

This produces an `Aligned` value: the same modules, each now carrying
a list of five record ids. The artifact's `alignment.module_to_records`
section is what `save` writes from this value.

The `K nearest records` knob ranges from 1 (only the nearest record
per module) to roughly module-size (every record in the module gets
listed). Sensible defaults: 5 for hand inspection, 50 for the
dispersion math, 100+ for production audits where the alignment
itself is a deliverable.

There is one extra knob, `fine label field is`:

```ocean static
align modules using 50 nearest records fine label field is primary_category
```

This tells `align` to also record, alongside each per-module record list, the fine-grained label for each of those records. The fine label field is typically a more granular version of the gold label. For the TNA corpus, the gold label is `archive` (two values: `bombe`, `tunny`) and the fine label might be `primary_category` (three values: `mechanical`, `electrical`, `structural`).

The fine label is not used by `find dispersion`. It is recorded for the human reader who later inspects the alignment section of the artifact and wants to see, at a glance, what kind of records each module collected.
```

find dispersion of each label

```ocean static find dispersion of each label

> ```
> `find dispersion of each label` is the verb that turns an `Aligned`
> value into a `Dispersion` value. The output is a single normalized
> score per label per module, indicating how cleanly each label is
> concentrated in or distributed across the modules.
>
> The artifact's `dispersion.by_label` section is the result.
>
> For the TNA pipeline with two archives (`bombe`, `tunny`) and six
> modules, the dispersion section might look like:
>
> ```json
> {
>  "dispersion": {
>    "by_label": {
>      "archive": {
>        "bombe": 0.71,
>        "tunny": 0.69
>      }
>    }
>  }
> }
> ```

A score of `1.0` means the entire `bombe` archive is concentrated in a single module, with no `bombe` records spread to other modules. A score of `0.0` means `bombe` records are spread perfectly evenly across every module, indistinguishable from random assignment. The 0.71 score above says the `bombe` archive concentrates in a small number of modules, but not in just one.

The exact formula is the normalized entropy of the per-module label proportions, inverted so that high values mean concentrated and low values mean spread. The normalization is against the number of modules in the artifact, so dispersion scores can be compared across runs with different `max_M_modules` settings.

What dispersion measures

Dispersion is the "did this actually find anything?" gauge. It answers a single question: do the modules that the unsupervised clustering produced happen to line up with the labels the data already had? The answer is one number per label, between 0 and 1.

Dispersion has three commercially load-bearing properties:

> **Single number.** A buyer asks "did this work?" and gets a number back. Not a confusion matrix, not a ROC curve, not a per-class table. One number per label. The substrate-status commitment depends on every customer being able to read the same gauge.
>
> **Deterministic.** Same seed, same corpus, same dispersion score to the bit. Two auditors running the same pipeline get the same number. This is what makes dispersion a contract rather than a suggestion.
```

Ungameable in one direction. A pipeline cannot "cheat" toward a high dispersion by, for instance, copying the gold labels into the embedding. The clustering operator never sees the labels; only `find dispersion` does, and it sees them only after the modules are fixed. A high dispersion score is therefore evidence of substrate structure, not of label leakage.

The fourth property, which the next section covers, is that dispersion can be *null-tested*.

Null testing

The substrate-clustering claim "the dispersion of `archive` is 0.71" is meaningful only if 0.71 is much higher than what random chance would produce on the same corpus. The null test answers that question by running the same clustering 500 times on a randomly shuffled version of the data and comparing the true score against the distribution of null scores.

The null test is not a verb in OCEAN. It is a separate operation in the primitive layer, documented in `docs/PRIMITIVE_SPEC.md` and Appendix C of this handbook. The reason it lives in the primitive layer is that the null test operates on fingerprints directly, without the embedding and clustering steps; it is much cheaper to run.

When a vendor publishes a dispersion claim ("0.71 on this corpus, z = 18.19σ against the null"), the second number is the null-test z-score. A claim with z below 2.0 is at chance level; above 3.0 is meaningful; above 5.0 is bulletproof for most commercial purposes.

Wider system

Dispersion is the falsifiability gauge that makes OCEAN-shaped infrastructure commercially viable. Without it, the substrate claim would be circular: "trust the modules because they came out of the algorithm." With it, the substrate claim is testable: "the modules concentrate label X with dispersion 0.71, which exceeds the 99th percentile of the null distribution, so the structure is real."

This is the same shape that makes p-values load-bearing in empirical science: a single number that can be checked against a known null. OCEAN's commercial commitment in `docs/PRIMITIVE_SPEC.md` section 11 is built on this: if a published claim ever fails the null test as specified, the vendor retracts the claim. That is the kind of commitment that only a falsifiable gauge makes possible.

Exercises

Add `narrate modules` between `align` and `find` in the Chapter 2 pipeline (chapter 12 covers narrate in detail). Does the dispersion value in the artifact change? Should it?

Run the Chapter 2 pipeline twice with the same seed and compare the dispersion values. Are they byte-identical? What if you change the seed to 43?

Read the dispersion score `0.69` for `tunny` in the example above. In your own words, what does it claim?

What's next

Chapter 9 covers `save` and the full determinism contract, including how the artifact's SHA-256 sidecar is computed and how to verify a re-run against a previous one.

Save and the Determinism Contract

> *After this chapter you understand exactly what 'deterministic' means for an OCEAN program, and why a re-run produces a byte-identical artifact.*

Concepts in this chapter

- What `save` writes (JSON + SHA-256 sidecar)
- The four pillars of OCEAN's determinism contract
- The seed and how to choose one
- How to verify that a re-run is identical

save and what it writes

`save` is the terminal verb of an OCEAN pipeline. It accepts any value of the four persistable types (`Records`, `Modules`, `Aligned`, `Dispersion`) and writes a JSON artifact to disk.

```ocean static
save to data/result.json
```

```
A single `save` writes two files. The first is the artifact itself,
a pretty-printed JSON document at the named path. The second is the
SHA-256 sidecar: a file with the same name plus a `.sha256`
extension, containing the hex digest of the artifact's bytes.
```

data/result.json
data/result.json.sha256

```
The sidecar is the verification contract. A reader who wants to
confirm an artifact matches the expected version computes
`sha256sum data/result.json` and compares to the sidecar's content. If they match, the artifact is
the right one. If they do not, something has changed.

`save` can be called multiple times in a single program. Each call
writes to its own path. There is no implicit final save; if a program
has no `save` statement, it runs to completion but produces no
on-disk output.

## The four pillars of determinism

OCEAN promises this: for any program _P_, any seed _S_, and any
input file with a fixed content, executing _P_ at seed _S_ produces
the same artifact and the same SHA-256 digest on every machine.

That promise rests on four pillars, each enforced by the compiler.

**Pillar 1: Inputs are content-addressed.** Every `load` line records
```

```
the SHA-256 of the file content into the operator's signature. If
the file content changes by a single byte, every downstream operator's
signature changes too, and the runner does not reuse cached
intermediates. The artifact records the source SHA-256 in
`pipeline.source_sha256`, so a reader can audit which input the
artifact corresponds to.

**Pillar 2: Operators are pure functions of inputs and seed.** No
operator reads wall-clock time, system entropy, or environment
variables for randomness. Where an operator would otherwise need
randomness (k-means initialization, BTUT sampling, random projection
seeding), it derives the randomness from `(input_signature, seed)`
and from nothing else.

**Pillar 3: Sweep expansion is order-stable.** A `sweep s from 42 to
45 do ... end` block always produces its four branches in ascending
order of `s`. Two readers running the same sweep collect their
results in the same sequence. This matters because two sweep
artifacts that are byte-identical when ordered ascendingly would
not be byte-identical if one runner happened to enumerate in
descending order.

**Pillar 4: Parallel does not cross state.** A `parallel do ... end`
block isolates its branches; they cannot share variables, write to
the same output path, or otherwise interact. This means the runner
can execute branches in any order without changing the artifact, and
each branch's output is reproducible on its own.

## The seed and how to choose one

```ocean static
seed 42
```

The `seed` declaration sets the integer seed used by every operator that needs randomness. The declaration is optional; if omitted, the runner uses 42 by default. The default is published as the industry-default reproducibility anchor in `docs/PRIMITIVE_SPEC.md`.

Three considerations for choosing a non-default seed:

Use a different seed when running a stability study. A `sweep seed from 42 to 45` produces four independent runs whose variation tells the reader how much the result depends on initialization. If all four runs land within rounding of each other, the result is stable. If they vary by more than a reasonable tolerance, the result is sensitive to initialization, and the report should say so.

Use a per-customer seed when a customer needs a unique artifact. This is rare but real for some compliance contexts; different customers get different artifacts so their files cannot be confused, but each customer's artifact is reproducible against their own seed.

Otherwise, leave it at 42. The default seed is the published anchor; a third party trying to reproduce a published claim should not have to guess which seed the publisher used.

How to verify a re-run is identical
```

The verification flow is two commands:

```
sha256sum data/result.json
cat data/result.json.sha256
```

Compare the two hex digests. If they match, the run is verified against itself. To verify across machines or across time, share the digest, the OCEAN program source, the seed declaration (if non-default), and the input file. A second party runs the same program at the same seed against the same input file and computes the SHA-256. The digests must match. If they do not, something has changed: the program text, the input file content, the operator versions in the runner, or the OCEAN compiler itself. The artifact's `pipeline.operator_versions` block helps narrow down which one.

There is no built-in `verify` verb in OCEAN. The verification is deliberately external: it uses standard POSIX tools, lives outside the program, and can be performed by a party who does not have OCEAN installed. This is the same shape that makes Git commit SHAs verifiable; the contract sits in the digest, not in the tool that produced it.

Wider system

Determinism is the commercial spine of OCEAN-shaped infrastructure. Without bit-identical artifacts, the falsifiability commitment in `docs/PRIMITIVE_SPEC.md` section 11 ("if a published claim ever fails the null test as specified, the vendor retracts the claim") cannot be tested. With bit-identical artifacts, the commitment is testable by anyone with the digest.

Comparison: Stripe's idempotency keys turn retried payments into deterministic operations; OCEAN's seed turns reruns of analytical pipelines into deterministic operations. In both cases, the underlying compute is fundamentally non-idempotent (a card authorization can fail or succeed; a clustering can hit different local minima), and the language-level guarantee is what makes the contract usable. Substrate status depends on this; an unreproducible substrate is not a substrate.

Exercises

Re-run the Chapter 2 pipeline twice with `seed 42`. Compute `sha256sum` of the artifact each time. The two digests should match. They do.

Re-run the same pipeline with `seed 43`. Does the dispersion value change a lot or a little? What does the size of the change tell you about stability?

The artifact has a `pipeline.operator_versions` block. If a future version of `cluster.kmeans` is shipped with a bug fix that changes the initialization in a small way, two runs against the old and new operator versions will produce different artifacts. How should the version field in the artifact help an auditor diagnose this?

What's next

Chapter 10 leaves the verb tour and covers control flow: `let`, `if`, `sweep`, `parallel`, and `compare`.

Control Flow

> *After this chapter you can write conditional pipelines, parameter sweeps, methodology comparisons, and parallel branches.*

Concepts in this chapter

- Named bindings with `let`
- Conditional branching with `if / elif / else`
- Parametric expansion with `sweep`
- Independent execution with `parallel`
- Methodology comparison with `compare ... against ... on ...`

Named bindings: let

`let` introduces a named binding. The right-hand side is any expression that produces a pipeline value or a primitive value.

```ocean static let raw = load tmp/corpus.ndjson take 500 records balanced by archive let z = embed text from raw into 128 dimensions let modules = cluster z for 16 rounds max 24 modules using kmeans

```
Each `let` binding has a type, inferred from the right-hand side.
The binding is in scope from its declaration to the end of the
enclosing block (which may be the whole program, or the body of a
`define`, `sweep`, `if`, or `parallel` block).

A binding can be referenced by name in any later statement. The
`from NAME` clause attached to `embed`, `cluster`, `align`, or `find`
selects which upstream binding to use:

```ocean static
let raw = load tmp/corpus.ndjson take 500 records balanced by archive
let z_tfidf = embed text from raw into 128 dimensions using tf-idf
let z_minilm = embed text from raw into 384 dimensions using transformer minilm_l6
```

Two embeddings on the same input. Both are in scope; downstream `cluster` statements can pick either one with `from z_tfidf` or `from z_minilm`.

Without `from NAME`, the compiler defaults to "most recent step of the expected type." This is convenient for short pipelines and brittle for long ones; the linter warns on `embed` statements that have multiple upstream `Records` candidates and no explicit `from`.

Conditional branching: if / elif / else

```ocean static if target > 5000 then reduce records using btut target 300 survivors budget $25 else embed text into 128 dimensions end

```
The condition is a Boolean expression. Both branches must produce
values of the same type, or one branch must be terminal (e.g., a
`return` inside a `define`).

`elif` chains additional conditions:

```ocean static
if target > 100000 then
   reduce records using btut target 1000 survivors budget $200
elif target > 5000 then
   reduce records using btut target 300 survivors budget $25
else
   embed text into 128 dimensions
end
```

The condition expression set is small: comparisons (`==`, `!=`, `<`, `>`, `<=`, `>=`), arithmetic (`+`, `-`, `*`, `/`), and the keyword booleans `and`, `or`, `not`. No function calls in condition positions (though `define`d functions can be invoked as separate statements).

Parametric sweeps

```ocean static sweep s from 42 to 45 do load *toy_corpora/toy_tna_50.ndjson take 50 records balanced by archive embed text into 64 dimensions using tf-idf cluster for 8 rounds max 6 modules using kmeans energy = corpus mean align modules using 5 nearest records find dispersion of each label save to data/seed*${s}.json end

```
A `sweep` expands at compile time into N independent branches, one
per value of the sweep variable. The body of the sweep is type-checked
once but executed once per value. The sweep variable is in scope
inside the body and is typically used to disambiguate output paths
via interpolation (`save to data/seed_${s}.json`).

The optional `step` clause sets a stride other than 1:

```ocean static
sweep d from 32 to 256 step 32 do
   embed text into d dimensions using tf-idf
   save to data/dim_${d}.json
end
```

Sweep expansion is deterministic. The branches always materialize in ascending order of the sweep variable. Two runs of the same sweep produce the same sequence of artifacts.

Independent execution: parallel

```ocean static parallel do save to data/tfidf_result.json save to data/minilm_result.json end
```

```
A `parallel` block declares that its statements have no inter-branch
dependencies and may be executed in any order, including concurrently.
The compiler verifies the independence by checking that no branch
references a binding produced by another branch.

`parallel` is rarely used in short pipelines; the compiler
automatically parallelizes independent branches of the DAG. The
explicit `parallel do ... end` is for cases where the reader wants
to communicate that two branches are intentionally independent and
should not share state.

## Methodology comparison: compare ... against ...

```ocean static
let raw = load _toy_corpora/toy_tna_50.ndjson take 50 records balanced by archive
compare
   embed text from raw into 64 dimensions using tf-idf
against
   embed text from raw into 64 dimensions using transformer minilm_l6
on dispersion of each label
```

`compare` runs two pipeline tails and reports the difference between their dispersion findings. The two arms must produce the same intermediate type (here, both produce `Z`), and the `on ...` clause names the finding the comparison is measured against.

The artifact for a compare has a slightly different shape than a single-pipeline artifact: it includes both arms' results plus a per-label delta:

```
{
 "compare": {
   "left": {"variant": "tf-idf", "dispersion": {...}},
   "right": {"variant": "transformer_minilm_l6", "dispersion": {...}},
   "delta": {"archive": {"bombe": -0.05, "tunny": +0.02}}
 }
}
```

A negative delta on `bombe` means the left arm's dispersion was lower than the right arm's; positive means the left was higher. The delta is the headline number; the two arm artifacts are kept for auditability.

`compare` is the right tool when a program author wants to know "does this methodological choice matter?" rather than "what is the result of one specific methodological choice?"

Wider system

OCEAN's control flow is deliberately restricted. There are no unbounded loops; no recursion (except in the operator-level TCD recursive loop, which is implementation detail); no goto; no exception handling. Every program compiles to a finite directed acyclic graph with a known branch count at compile time.

This is the same restriction SQL imposes and for the same reason: a finite DAG with known branches is something a planner can reason about. The cost shape is predictable. The artifact set is enumerable. The audit trail is bounded. Substrate-status infrastructure depends on these properties; a
```

Turing-complete OCEAN would be a research language, not a substrate.

Exercises

Write a sweep that runs the toy_tna_50 pipeline at dimensions 32, 64, 128, and 256. Save each to a separate artifact path.

Write a `compare` that puts `tf-idf` against `one-hot numeric` on the toy_nslkdd_200 corpus. Read the delta. Which embedder gives higher dispersion on the `type` label? Why?

Sketch the type error that the compiler would produce for the following snippet:

```
ocean static if target > 1000 then embed text into 128 dimensions else cluster for 16 rounds max 24 modules using kmeans end
```

(Hint: the two arms of an `if` must produce the same type.)

What's next

Chapter 11 covers user-defined functions: `define`, default parameters, and the standard library at `scripts/operators/ocean/stdlib/substrate.ocean`.

Functions, Modules, Stdlib

> *After this chapter you can read the standard library, call its functions, and write your own.*

Concepts in this chapter

- `define` for naming a reusable pipeline body
- Default parameters and named-argument call syntax
- `import "path" as namespace` for cross-file reuse
- The substrate stdlib at `scripts/operators/ocean/stdlib/substrate.ocean`
- When to author a stdlib function versus an inline pipeline

Defining a function

```ocean static define basic_run(corpus, target = 500, embed_dim = 128, iters = 16, k_nearest = 50, output = "data/validation/basic_run.json") do load corpus take target records balanced by archive embed text into embed_dim dimensions cluster for iters rounds max 24 modules using kmeans energy = corpus mean align modules using k_nearest nearest records find dispersion of each label save to output end

> `define NAME(params) do ... end` introduces a reusable, named
> pipeline. The body is any sequence of statements legal at top level.
> Parameters appear as identifiers usable in the body. Parameters with
> default values are optional at call sites.
>
> The function above takes one required parameter (`corpus`) and five
> optional ones with defaults. Call sites supply the corpus and
> override any default they want:
>
> ```ocean static
> basic_run(corpus = "_toy_corpora/toy_tna_50.ndjson", target = 50)
```

Calls use the named-argument syntax. Positional arguments are also legal but discouraged for anything beyond the first one or two parameters; named arguments read better and are less brittle to function-signature changes.

Default parameters and types

Parameter types are inferred from their default values, when present, or annotated explicitly:

```ocean static define run_at_seed(corpus : Path, target : Number = 500) do ... end
```

````
When the type is unambiguous from the default (a number literal, a
path literal, a string literal), the annotation can be omitted. When
a parameter has no default and is called with mixed types across
sites, annotate explicitly.

The compiler enforces parameter types at call sites: passing a
string to a parameter annotated as `Path` is a type error.

## return: making functions reusable as expressions

A `define`d function can `return` a value, making it usable on the
right-hand side of `let`:

```ocean static
define embed_and_cluster(corpus, dim = 128) do
   let raw = load corpus take 500 records balanced by archive
   let z = embed text from raw into dim dimensions using tf-idf
   let modules = cluster z for 16 rounds max 24 modules using kmeans
   return modules
end
````

```ocean static let m = embed_and_cluster(corpus = "tmp/x.ndjson", dim = 64) align m using 50 nearest records find dispersion of each label

````
The return type is inferred from the type of the returned value (here,
`Modules`).

Imports

```ocean static
import "presets/atlas.ocean" as atlas
let result = atlas.basic_run(corpus = "_toy_corpora/toy_tna_50.ndjson")
````

`import "PATH" as NAME` reads the named file, parses and type-checks it independently, and exposes its top-level `define`s under the namespace `NAME`. Only `define`d functions are visible across the import; top-level statements in the imported file are not executed.

Imports are textual and namespaced. Cyclic imports are an error. Two imports with the same `as NAME` are an error. The path is relative to the importing file, or relative to a configured include root.

The substrate stdlib

The standard library ships at `scripts/operators/ocean/stdlib/substrate.ocean`. It defines four common pipelines as `define`d functions:

`basic_run(corpus, target, embed_dim, iters, k_nearest, output)` runs the simplest end-to-end pipeline. The defaults are tuned for a 500-record sample: 128 dimensions, 16 rounds, 24 modules, 50 nearest records.

`seed_sweep(corpus, target, first_seed, last_seed)` runs the same pipeline at every seed in the inclusive range and saves to `data/validation/seed_${s}.json`. The default range is 42 to 45, producing four artifacts to compare.
````

`anomaly_focused(corpus, normal_label, target, output)` runs an anomaly-detection pipeline with the energy function anchored on the normal class. Defaults match the NSL-KDD intrusion-detection corpus shape (`normal` as the normal label, `type` as the label field).

`content_vs_structural(corpus, output)` runs a `compare` between TF-IDF and the premium content fingerprint, measured on dispersion of the `directorate_to_pm` label. This preset requires an API key for the content fingerprint operator; without one, the compare type-checks but the content arm does not execute.

To call any preset from a program:

```ocean static import "stdlib/substrate.ocean" as substrate

let result = substrate.basic_run( corpus = "_toy_corpora/toy_tna_50.ndjson", target = 50 ) ```

## When to author a stdlib function

The right time to extract a preset into the stdlib is when the same pipeline shape has appeared in two or more programs in the project, and the parameters that vary across sites are bounded.

A pipeline shape that is unique to one program should stay inline. A pipeline shape that varies on every call should not become a stdlib function; it should remain a directly-written pipeline so the variation is visible.

Stdlib functions in OCEAN are presets, not abstractions. They name a sensible-default pipeline and let the call site override the few parameters that typically vary. A preset that hides every parameter behind a wall of defaults is a code smell; readers should be able to read the preset's signature and know roughly what it does.

## Wider system

The stdlib is the most concrete piece of the substrate-status play. Every preset (`basic_run`, `seed_sweep`, `anomaly_focused`, `content_vs_structural`) is a pattern that customers describe in their own work even if they never import the stdlib. When a customer team reports "the seed sweep showed stable dispersion," they have already adopted OCEAN's vocabulary; whether they ran `substrate.seed_sweep` or hand-wrote the `sweep` statement is a detail.

This is the same shape that made dbt's macros load-bearing for analytics engineers: the named patterns became the vocabulary, and the vocabulary became the way the work was described, and the way the work was described became how the rest of the toolchain organized itself.

## Exercises

> Call `substrate.seed_sweep` against the toy_tna_50 corpus with `first_seed = 42` and `last_seed = 43`. How many artifacts does the call produce?
```

Define a function `my_anomaly(corpus)` that wraps `anomaly_focused` with `target = 100` and a fixed output path. Call it.

The stdlib has `basic_run`, `seed_sweep`, `anomaly_focused`, and `content_vs_structural`. Suggest one more preset that would be a good candidate for the stdlib, and write a one-line description of its parameters.

What's next

Chapter 12 covers the tooling: the REPL, the formatter, the linter, the LSP, and the OCEAN MCP server that exposes the language as a set of tools for an AI coding agent.

Tooling and the LSP

> *After this chapter you can compile, run, format, lint, and edit OCEAN with full IDE support, and you know how to expose OCEAN as a tool to an AI coding agent.*

Concepts in this chapter

- The compile-and-run CLI
- The REPL for incremental exploration
- The formatter (canonical pretty-printing)
- The linter (style and dead-code analysis)
- The Language Server Protocol implementation
- The Model Context Protocol server (`ocean-mcp`)
- The `narrate` verb for human-readable annotations

Compile and run

The single entry point for compiling and running an OCEAN program:

```
python -m scripts.run_universal_pipeline --config first_pipeline.ocean
```

The runner parses, type-checks, and executes the program. Step timings appear on stdout in the form `[step 1/N] verb ... Xms · summary`, which is the format the sandboxed handbook runner parses for its output panel. Failures produce typed diagnostics with line and column information.

The runner accepts a `--seed N` flag that overrides any `seed` declaration in the program, useful for `sweep` interactions and batch reruns. The flag is also useful for stress-testing whether a result is stable across seeds without editing the source file.

REPL: interactive exploration

```
python -m scripts.operators.ocean.repl
```

The REPL is a stateful line-by-line interpreter. Each line is parsed and type-checked; if successful, it is executed and any resulting bindings persist to the next line.

```
ocean> let raw = load _toy_corpora/toy_tna_50.ndjson take 50 records balanced by archive
:: Records (50 records, source_sha256=a1b2c3...)

ocean> let z = embed text from raw into 64 dimensions
:: Z (shape: 50 x 64)
```

```
ocean> let m = cluster z for 8 rounds max 6 modules using kmeans
:: Modules (4 modules; sizes: [15, 13, 12, 10])
```

The REPL is the right tool for exploring a new corpus or for prototyping a pipeline shape before committing to a `.ocean` file. Each successful evaluation displays the resulting value's type and a one-line summary. The actual contents of values are still opaque; the REPL does not let the user read inside a `z` value.

State persists across lines for the duration of the session. A `:reset` command clears the binding namespace. A `:save FILE` command writes the current sequence of bindings as a `.ocean` program.

The formatter

```
python -m scripts.operators.ocean.format file.ocean
python -m scripts.operators.ocean.format file.ocean --write
```

Without `--write`, the formatter prints the canonical form to stdout. This is useful for diff-style checks: `diff <(format file.ocean) file.ocean` shows any non-canonical formatting.

With `--write`, the formatter overwrites the file in place. This is the right setup for editor-on-save hooks and for pre-commit hooks in the repository.

The canonical form fixes whitespace (one space around `=`, no trailing whitespace, single blank line between top-level statements), normalizes literal forms (`tfidf` becomes `tf-idf`, numeric underscores are added for groupings above 1000), and sorts named arguments alphabetically inside a call.

The linter

```
python -m scripts.operators.ocean.lint file.ocean
python -m scripts.operators.ocean.lint file.ocean --strict
```

The linter catches style and dead-code issues that the typechecker does not. The default lint level is informational warnings:

- Unused `let` bindings.
- Unused imports.
- Missing `seed` declaration (the default is 42, which is fine, but the linter prefers an explicit seed for reproducibility contracts).
- Missing `label field is` declaration when the default `archive` is not a field in the corpus.
- Sweeps with single-step iterations (`sweep s from 42 to 42`).
- Cluster statements with no upstream embed.
- Paths that do not exist on disk at lint time.
- Magic-number dimensions outside the common set `{32, 64, 128, 192, 256}`.

With `--strict`, warnings become errors. The pre-commit hook in the repository runs the linter with `--strict` on staged `.ocean` files; new programs cannot be committed with sweeps over a single value or with paths that do not exist.

The Language Server Protocol

```
python -m scripts.operators.ocean.lsp
```

The LSP implementation runs the parser, type-checker, and linter incrementally on each keystroke and reports diagnostics back to the editor via the standard LSP wire protocol. Hover-tooltips show the type of any binding. Go-to-definition works for `define`d functions and imports.

Editor configurations live in the repository under `editor/vscode/` and `editor/cursor/`. The Goose configuration ships with the `ocean-mcp` package and registers OCEAN under `~/.config/goose/config.yaml`.

Model Context Protocol: OCEAN as an agent tool

```
pip install ocean-mcp
ocean-mcp                                    # runs as a server on stdio
```

The OCEAN MCP server exposes the language as a set of tools for an AI coding agent: `ocean_compile`, `ocean_validate`, `ocean_run`, `ocean_format`, `ocean_lint`, `ocean_list_ops`, `ocean_list_stdlib`. The tool surface is documented in `packages/ocean-mcp/README.md`.

Configuration for the major MCP-compatible clients:

```
{
 "mcpServers": {
   "ocean": {
     "command": "ocean-mcp"
   }
 }
}
```

For Claude Desktop on macOS, place this in `~/Library/Application Support/Claude/claude_desktop_config.json`. For Cursor, add via Settings to MCP. For Goose, edit `~/.config/goose/config.yaml`.

With the MCP server running, an agent can ask "validate this OCEAN snippet" and get back the same diagnostics a human would see in their editor. This closes the loop between agent-authored programs and the language's safety guarantees: an agent that writes a type-incorrect OCEAN program gets a typed error and can fix it.

The narrate verb

`ocean static narrate modules in technical style`

`narrate` is the last unmentioned verb. It takes a `Modules` or `Aligned` value and attaches human-readable narrative strings to each module by inspecting the module's centroid and its closest records. The output type is the same as the input (`Modules` becomes `Modules`, `Aligned` becomes `Aligned`), with the `narrative` slot on each module now populated.

Three styles are accepted: `technical` (operator-and-parameter focused), `plain` (label-and-record focused), `terse` (one-line per module). The default is `plain`.

`narrate` is usually the last verb before `save`. Run it after `align` so the narrative can reference specific records. Do not run it between `cluster` and `align`; the type system allows it (narrate accepts `Modules`), but the narratives produced from bare modules without record alignments are less informative.

Wider system

A tiny DSL with its own LSP and its own MCP server is disproportionate effort unless the goal is substrate status. The first time an agent autocompletes `cluster for 16 rounds max 24 modules`, OCEAN has won a token-level adoption point. The first time an editor's hover-tooltip shows `Z (shape: 500 x 128)` without the program author having to think about types, OCEAN has won a comprehension point.

Substrate status is the accumulation of these small wins across millions of interactions. The tooling exists to make those wins free.

Exercises

Run the linter on the Chapter 2 pipeline. What does it warn about, if anything?

Start the REPL and create two let-bindings, the second referencing the first. Confirm the second binding's displayed type matches what Chapter 4's type table predicts.

Install the MCP server (or read its README), and write a one-line description of what `ocean_list_ops` would return for the version of OCEAN this handbook documents.

What's next

Chapter 13 covers patterns that experienced OCEAN authors use and patterns that beginners often write that turn out badly.

Effective OCEAN

> *After this chapter you write OCEAN the way an experienced OCEAN author writes it.*

Concepts in this chapter

- Five idioms worth internalizing
- Four anti-patterns to avoid
- When to reach for which embedder, clusterer, and energy function

Idiom 1: name your bindings

Inline pipelines that flow one verb into the next are convenient for prototyping. For anything beyond two or three verbs, name the intermediate values with `let`:

```ocean static let raw = load _toy_corpora/toy_tna_50.ndjson take 50 records balanced by archive let z = embed text from raw into 64 dimensions using tf-idf let m = cluster z for 8 rounds max 6 modules using kmeans let aligned = align m using 5 nearest records find dispersion of each label from aligned

```
The named version is one or two lines longer than the implicit
version, but it has three properties the implicit version lacks.
The reader can see at a glance what each step produces. A linter
warning that fires on "ambiguous upstream binding" has somewhere
to point. The same `raw` or `z` can be reused in a later `compare`
without re-running the upstream operators.

Idiom 2: small seeds before sweeps

Before running a `sweep` over many values, confirm the single-value
path works:

```ocean static
seed 42
load _toy_corpora/toy_tna_50.ndjson take 50 records balanced by archive
embed text into 64 dimensions using tf-idf
cluster for 8 rounds max 6 modules using kmeans
align modules using 5 nearest records
find dispersion of each label
save to data/seed42.json
```

Run this first. If the artifact has the right shape, then wrap the same body in `sweep s from 42 to 45 do ... end` and re-run. This catches a class of bugs where the single-value path was wrong but the sweep masked it across four runs of identical broken output.

Idiom 3: prefer compare over duplicate pipelines

When the question is "does methodology A produce a different dispersion than methodology B," use `compare`, not two separate pipelines with manual diffing:

```ocean static let raw = load _toy_corpora/toy_tna_50.ndjson take 50 records balanced by archive compare embed text from raw into 64 dimensions using tf-idf against embed text from raw into 64 dimensions using transformer minilm_l6 on dispersion of each label
```

```
The `compare` form ensures both arms run against the same `raw`
binding, on the same seed, with the same surrounding parameters.
A hand-written diff between two `save to ...` artifacts misses any
of those equalities that the author forgot to keep in sync.

## Idiom 4: narrate last, never between cluster and align

```ocean static
let m = cluster z for 16 rounds max 24 modules using kmeans
let aligned = align m using 50 nearest records
let narrated = narrate aligned in plain style
find dispersion of each label from narrated
save to data/result.json
```

`narrate` accepts both `Modules` and `Aligned` (since `Aligned <: Modules`), so the compiler will not complain if a program runs it on bare `Modules` before alignment. Resist that. Narratives produced from bare modules name only the centroid, not the records, and read as generic. Run `narrate` on `Aligned`, after `align`, so the narrative can reference specific records.

Idiom 5: explicit seed every time

```ocean static seed 42
```

```
Even when 42 is the default, write the declaration. A future
maintainer who reads the source should not have to remember the
default. An auditor who reads the artifact's `pipeline.seed` field
should be able to point at the source line that set it.

The linter warns on programs without an explicit `seed`. Treat that
warning as binding.

## Anti-pattern 1: re-embed per sweep branch

```ocean static
# Wrong: re-embeds 4 times.
sweep s from 42 to 45 do
   load _toy_corpora/toy_tna_50.ndjson take 50 records balanced by archive
   embed text into 64 dimensions using tf-idf
   cluster for 8 rounds max 6 modules using kmeans
   save to data/seed_${s}.json
end
```

The `embed` step is deterministic; with the same input file and the same seed strategy (TF-IDF embedding does not depend on the clustering seed), it produces the same `Z` on every iteration. Re-running it inside the sweep wastes compute.

The right shape is to embed once, then sweep the seeds that matter:

```ocean static let raw = load *toy_corpora/toy_tna_50.ndjson take 50 records balanced by archive let z = embed text from raw into 64 dimensions using tf-idf sweep s from 42 to 45 do cluster z for 8 rounds max 6 modules using kmeans save to data/seed*${s}.json end

```
The clustering operator picks up the loop variable as its seed via
the standard seed-derivation contract (the linter warns if the
sweep variable is never referenced by the body).

Anti-pattern 2: hidden upstream dependencies

```ocean static
# Wrong: which Records is align using?
load tmp/train.ndjson take 500 records balanced by label as train_records
load tmp/eval.ndjson take 100 records balanced by label as eval_records
embed text into 128 dimensions from train_records
cluster for 16 rounds max 24 modules using kmeans
align modules using 50 nearest records
```

The `align` step has two candidate `Records` bindings (`train_records` and `eval_records`). The compiler defaults to "most recent of the required type," which is `eval_records`. This is almost certainly not what the program author intended; modules built from training embeddings should not be aligned against the eval set without the author saying so explicitly.

The fix is an explicit `from` clause:

```ocean static align modules using 50 nearest records from train_records

```
The linter warns whenever an implicit upstream binding has more than
one candidate. Treat that warning as binding.

Anti-pattern 3: tuning dimensions before checking dispersion

A common beginner instinct: see a dispersion of 0.4, decide it is
too low, double the embedding dimension, re-run. The dispersion goes
up to 0.5, but the modules are now full of records that are
spuriously similar in the new high-dimensional space. The
dispersion improved, but the substrate did not.

The right move is to check the null test first. A dispersion of 0.4
with a null mean of 0.39 means the modules are at chance level; no
amount of dimension tuning will help. A dispersion of 0.4 with a
null mean of 0.10 means the modules already capture real substrate;
the question is whether 0.4 is enough for the use case, not whether
0.5 is achievable.

The null test lives at the primitive level (Appendix C); the
handbook does not have a verb for it because it is corpus-level
audit, not pipeline-level work.

Anti-pattern 4: copying the gold label into the embedding

The most insidious anti-pattern: a program author looking at a low
dispersion score adds a feature to the embedding that derives from
```
```

```
the gold label.

```ocean static
# Wrong: text field now contains the gold label.
# The embedder sees archive='bombe' as a token, indistinguishable
# from a content word. Dispersion will be artificially high.
```

This breaks the falsifiability of the substrate claim. The dispersion finding is now a tautology; the modules separate the gold labels because the embedder knew the gold labels. The remedy is to confirm at the corpus-construction level that the text and label fields are independent: a feature derived from the label cannot appear in the text.

This is the substrate-clustering analog of label leakage in supervised learning. OCEAN's design helps avoid it by separating the `text field is` and `label field is` declarations, but the field contents are out of OCEAN's control. A diligent program author checks the corpus.

When to reach for what

A quick decision tree, for a reader who has read the previous twelve chapters and now needs to write one.

Choosing the embedder. - Text corpus, English, vocabulary matters: `tf-idf`. - Text corpus, multilingual or paraphrase-dominant: `transformer minilm_l6`. - Non-text corpus (network logs, telemetry): `one-hot numeric`. - Structural similarity across surface differences: `content fingerprint` (premium).

Choosing the clusterer. - Prototyping a new pipeline: `kmeans`. - Production run with substrate guarantees: `tcd recursive loop` (premium).

Choosing the energy function. - No privileged class: `energy = corpus mean`. - A well-defined normal class: `energy = normal anchored on LABEL`.

Choosing the alignment width. - Hand-inspecting modules: `using 5 nearest records`. - Dispersion math: `using 50 nearest records`. - Production audit: `using 100 nearest records` or more.

Wider system

This chapter is where readers learn the culture of OCEAN, not the syntax. A language acquires substrate status partly through a shared sense of which patterns are "the right way." The idioms above are the patterns that experienced OCEAN authors converge on independently; the anti-patterns are the patterns that show up in code review.

The substrate-status angle: when a customer's data team adopts these idioms unprompted, OCEAN has won. The vocabulary is not just the verb list; it is the shared mental model of which way of writing the verb list is the right way.

Exercises
```

1. Take this anti-pattern snippet and rewrite it into the idiomatic form, naming bindings appropriately:

```
ocean static load tmp/x.ndjson take 500 records embed text into 128 dimensions using tf-idf cluster for 16 rounds max 24 modules using kmeans align modules using 50 nearest records find dispersion of each label save to /tmp/out.json
```

   Of the four anti-patterns in this chapter, which one is the hardest to catch in code review and why?

   Write a one-paragraph mental rule for when to wrap a pipeline in a `sweep` versus when to leave it as a single-value run.

What's next

Chapter 14 covers interfacing OCEAN to the rest of a system: Postgres extension, HTTP API, MCP server, CLI, agent loops.

Interfacing OCEAN

> *After this chapter you can invoke OCEAN from Postgres, the HTTP API, the MCP server, the CLI, and an agent loop.*

Concepts in this chapter

- The `pg_latentocean` Postgres extension
- The HTTP API for remote pipeline execution
- The OCEAN MCP server as an agent tool surface
- The CLI for batch usage
- The agent-loop pattern: compose, validate, run, iterate

pg_latentocean: OCEAN inside Postgres

The Postgres extension `pg_latentocean` adds three functions to a Postgres database:

```
SELECT lo_fingerprint(row_to_json(t)) AS fp
FROM   submissions t
WHERE  t.created_at > now() - interval '7 days';
```

`lo_fingerprint(JSONB)` returns the 48-bit structural fingerprint of the row, as a `bit(48)` value. The fingerprint is the same one the OCEAN `embed.content_fp48` operator produces; the contract is in `docs/PRIMITIVE_SPEC.md`.

```
SELECT * FROM lo_score_window('submissions', 'composite', 100);
```

`lo_score_window(table, dim, window)` returns the top-N rows of a table ranked by the named score dimension, where `dim` is one of `composite`, `anomaly`, `reconstruction`, or `diversity`. The window argument bounds the rolling history that scores are computed against.

```
SELECT lo_null_test('submissions', '{composite}', 500, 42);
```

`lo_null_test(table, dims, iterations, seed)` runs the null-test operation defined in `docs/PRIMITIVE_SPEC.md` section 4. The return value is a `lo_null_result` composite type with the headline z-score and the per-quantile null distribution.

The Postgres extension is the right surface when the data is already in Postgres and the analyst's first language is SQL. It runs the fingerprint primitive directly in the database; no data leaves the server. For full pipeline operations (`embed`, `cluster`, `align`, `find`), the SQL surface returns an OCEAN program text that the analyst then runs through the compiler or the HTTP API.

The HTTP API

```
POST /api/v1/run
Content-Type: application/json
Authorization: Bearer lo_pk_...

{
 "source": "<ocean program text>",
 "seed": 42
}
```

The HTTP API accepts an OCEAN program, runs it server-side, and returns the artifact in the response body. The server handles file-corpus uploads via a separate `/api/v1/corpora` endpoint that returns a corpus id usable in `load` statements.

```
GET /api/v1/runs/{run_id}/artifact
GET /api/v1/runs/{run_id}/artifact.sha256
```

Each successful run produces a stable `run_id`. The artifact and its SHA-256 sidecar are addressable forever (subject to retention policy) via these endpoints. The artifact's `pipeline.source_sha256` field is the canonical reference for "what corpus did this run see;" the run_id is the canonical reference for "which specific execution of which specific program."

The HTTP API is the right surface when OCEAN runs on a remote server (a customer's own infrastructure, a managed Latent Ocean deployment) and the caller is anything other than a Postgres database: a notebook, a webhook, a CI job.

MCP: OCEAN as an LLM tool

The MCP server (`ocean-mcp`, covered in chapter 12) exposes the language as a set of tools for an AI coding agent. The full tool list:

Tool	What it does
`ocean_compile`	Compile source to operator-DAG, no execution.
`ocean_validate`	Type-check only, return diagnostics.
`ocean_run`	Compile and execute end-to-end.
`ocean_format`	Pretty-print to canonical form.
`ocean_lint`	Style and dead-code warnings.
`ocean_list_ops`	Enumerate operators with English schemas.
`ocean_list_stdlib`	List stdlib presets.

An agent equipped with these tools can:

1. Receive a natural-language request ("cluster these 5,000 SEC filings and tell me which ones are anomalies").
2. Look up the available operators via `ocean_list_ops`.
3. Compose an OCEAN program using those operators.
4. Run `ocean_validate` to confirm the program type-checks.
5. Run `ocean_run` to execute it and read the artifact back.

The agent loop is the substrate-status play in real time. Every agent that adopts these tools is one more place where the OCEAN vocabulary lives in the agent ecosystem.

The CLI

```
ocean compile file.ocean              # parse + typecheck + emit DAG
ocean run file.ocean                  # compile + execute
ocean fmt file.ocean --write          # pretty-print in place
ocean lint file.ocean --strict        # style + dead-code
ocean repl                            # interactive
ocean version                         # toolchain version
```

The CLI is a thin wrapper over the same Python modules covered in chapter 12. It is the right surface for shell-script usage and for CI pipelines.

The `ocean run` command exits with status 0 on success, 1 on compile/type error, 2 on runtime error. Diagnostic output goes to stderr; step timings and the artifact path go to stdout. This makes the CLI composable with the standard Unix tool stack:

```
ocean run pipeline.ocean | jq '.dispersion.by_label' | grep -v '^null'
```

The agent-loop pattern

The "agent loop" is the pattern that all five interfaces above participate in:

1. **Compose.** The agent (human, LLM, or hybrid) writes an OCEAN program based on the question being asked.
2. **Validate.** The program is type-checked via `ocean_validate` or `ocean compile`. Type errors come back as typed diagnostics with suggestions. The agent fixes them.
3. **Run.** The program executes, either locally (CLI), via the HTTP API, or through MCP.
4. **Read.** The artifact comes back. The agent inspects the `dispersion` section, the `modules` section, and the `pipeline.operator_versions` block.
5. **Iterate.** Based on the artifact, the agent adjusts the program (different dimensions, different energy function, different alignment width) and re-runs.

The loop is short because OCEAN's compile-and-typecheck step is fast (under a second on programs up to a few hundred lines). The loop is robust because every iteration produces a saved artifact with a

deterministic digest; if the agent's third iteration was the right one, the artifact is the contract.

Wider system

The five interfaces above are the deployment-surface list. Each surface lowers the activation energy for a new user to find OCEAN already inside the tool they were going to use anyway. A Postgres user does not need to install anything new; an analyst with an HTTP client can run a pipeline; a Claude Desktop user gets OCEAN tools surfaced automatically once the MCP server is configured.

This is comparable to how `psql` ships in every Linux distribution and how `grep` shipped in every Unix variant; ubiquity is the moat. Substrate status is built one deployment surface at a time, with the same verb namespace on every one.

Exercises

Write a `psql` query that calls `lo_fingerprint` on the first five rows of a hypothetical `submissions` table and returns the id and the fingerprint side by side.

Configure the OCEAN MCP server in a Claude Desktop or Cursor installation. Verify by asking the agent to `ocean_list_ops`.

Sketch an agent loop, in five bullet points, for the question "are there structural outliers in the trade-flow data from the last quarter?" Reference at least three of the interfaces from this chapter.

What's next

The appendices: the grammar (A), the operator catalog (B), the primitive spec companion (C), the glossary (D), the reference card (E), and the exercise solutions (F).

Appendix A: Grammar

> *This appendix is the complete EBNF grammar of OCEAN 1.0.*

The grammar below is identical to `docs/OCEAN_LANG.md` section 2. It is normative; any disagreement between this appendix and the formal reference is a bug against this handbook, not against OCEAN. Terminals are quoted; non-terminals are bare.

EBNF

```
program        = [ require_decl ] [ seed_decl ] { import_stmt } { top_stmt } EOF ;

require_decl   = "require" "ocean" version ;
version        = int "." int [ "." int ] ;

seed_decl      = "seed" int ;

import_stmt    = "import" string [ "as" ident ] ;

top_stmt       = define_decl | statement ;

define_decl    = "define" ident [ "(" param_list ")" ]
                 "do" { statement } "end" ;
param_list     = param { "," param } ;
param          = ident [ ":" type_expr ] [ "=" literal ] ;

statement      = let_stmt
               | sweep_stmt
               | compare_stmt
               | parallel_stmt
               | if_stmt
               | return_stmt
               | verb_stmt ;

let_stmt       = "let" ident [ ":" type_expr ] "=" expression ;

sweep_stmt     = "sweep" ident "from" int "to" int [ "step" int ]
                 "do" { statement } "end" ;

compare_stmt   = "compare" expression "against" expression
                 [ "on" ident "of" ident ] ;

parallel_stmt  = "parallel" "do" { statement } "end" ;

if_stmt        = "if" expression "then" statement
                 { "elif" expression "then" statement }
                 [ "else" statement ]
                 "end" ;

return_stmt    = "return" expression ;

verb_stmt      = verb { phrase } [ "as" ident ] [ "using" variant ] ;

phrase         = phrase_kw value_expr
```

```
              | "from" ident
              | int ;

verb            = "load" | "embed" | "reduce" | "cluster" | "align"
              | "find" | "narrate" | "save" ;

variant         = ident { ident } ;

expression      = literal
              | ident
              | verb_stmt
              | call_expr
              | binary_expr
              | "(" expression ")" ;

call_expr       = ident "(" [ expression { "," expression } ] ")" ;

binary_expr     = expression bin_op expression ;
bin_op          = "==" | "!=" | "<" | ">" | "<=" | ">="
              | "+"  | "-"  | "*" | "/"
              | "and" | "or" ;

literal         = int | float | string | path | bool | interp ;

type_expr       = type_name [ "[" type_expr "]" ] ;
type_name       = "Records" | "Z" | "Modules" | "Aligned" | "Dispersion"
              | "Artifact" | "Pipeline" | "Number" | "String"
              | "Path" | "Bool" | "Any" ;
```

Lexical productions (informal)

```
ident   ::= [a-z_] [a-z0-9_]*
int     ::= [-]? [0-9] ([0-9_]* [0-9])?
float   ::= [-]? [0-9]+ ('.' [0-9]+)? ([eE] [+-]? [0-9]+)?
string  ::= '"' (any-char-except-quote | escape)* '"'
         |  "'" (any-char-except-quote | escape)* "'"
escape  ::= '\' ('"' | "'" | '\' | 'n' | 't' | 'r')
path    ::= [\w./\\-]+ '.' ('ndjson' | 'csv' | 'tsv' | 'json'
                          | 'yaml' | 'yml' | 'txt' | 'ocean')
bool    ::= 'true' | 'false'
interp  ::= '${' ident '}'
comment ::= '#' (any-char-except-newline)*
```

Newlines are statement-significant outside parenthesized expressions and brace blocks. Whitespace separates tokens.

Reserved words

```
require    seed       let        in         as         on         from
to         into       using      with       by         of         do
end        sweep      compare    against    parallel   import     step
take       balanced   field      is         for        rounds     round
max        modules    module     energy     crystallize every     nearest
records    record     dispersion each       label      fine       anchored
```

```
dimensions dimension  loop       recursive  tcd        tf-idf     tfidf
content    fingerprint one-hot   numeric    mean       corpus     normal
text       define     return     if         then       else       elif
true       false      not        and        or
```

Verbs

```
load    embed    reduce    cluster    align    find    narrate    save
```

Operators

```
=  ,  {  }  (  )  ==  !=  <  >  <=  >=  +  -  *  /
```

Boolean operators are spelled `and`, `or`, `not`. There are no bitwise operators, no modulo, no increment or decrement.

Appendix B: Operator Catalog

> *Every operator OCEAN ships, with its English-labeled parameter schema, plus the three bundled toy corpora.*

How to read this appendix

The catalog table below names every operator the OCEAN runtime ships, in name-alphabetical order. The Tier column says whether the operator runs in the free-tier sandbox (`free`) or requires a paid API key (`premium`). The Signature column names the OCEAN type the operator consumes and produces.

The catalog table is generated by `scripts/handbook/build.py` from the registry at `backend/handbook_runner/premium_gate.py`. The prose around the table is hand-written; the table itself updates automatically when the registry changes. Any drift between the registry and the generated table is a CI failure.

Open-core operators

These operators run in the free-tier sandbox. The reader of this handbook can invoke any of them from a `.ocean` source file or via the inline Run button on a runnable snippet.

Operator	Tier	Signature	Summary
`align.module`	free	`(Modules, Records, Z) -> Aligned`	Align modules to records via k-nearest neighbors.
`cluster.kmeans`	free	`Z -> Modules`	Standard k-means clustering with deterministic initialization.
`embed.tfidf_jl`	free	`Records -> Z`	TF-IDF embedding followed by Johnson-Lindenstrauss random projection to D dimensions.
`embed.transformer.minilm_l6`	free	`Records -> Z`	MiniLM-L6 sentence-transformer embedding at 384 dimensions native.
`find.dispersion_per_label`	free	`(Aligned, Records, Z) -> Dispersion`	Compute the dispersion of each label across modules.

`load.ndjson`	free	`Path -> Records`	Load an NDJSON corpus from disk, optionally stratifying the sample.
`persist.json`	free	`Any -> Artifact`	Write the input value as pretty-printed JSON to disk, with a sha256 sidecar.

Premium operators

These operators are parsed and type-checked by the free-tier compiler, so the diagnostics elsewhere in a program remain accurate. Execution requires a paid API key (`OCEAN_API_KEY`).

Operator	Tier	Signature	Summary
`align.dispersion`	premium	`(Modules, Records, Z) -> Aligned`	Dispersion-weighted alignment (premium alignment with module-quality scoring).
`cluster.tcd_recursive_loop`	premium	`Z -> Modules`	TCD recursive-loop clustering with monotone module-energy guarantees (premium algorithm).
`embed.content_fp48`	premium	`Records -> Z`	Bloom-style 48-bit content fingerprint over top-K terms (premium structural primitive).
`reduce.btut`	premium	`(Z, Records) -> (Z, Records)`	BTUT structural-anomaly pre-reduction targeting N survivors within a compute budget.

Get an API key at https://latentocean.com/protocols.

Toy corpora

The handbook ships three small NDJSON corpora at `backend/handbook_runner/corpora/`. Each is small enough to run a full pipeline in under five seconds on a laptop. Snippets in the handbook

reference them by short name; the runner stages the corresponding file into a sandboxed working directory at run time.

toy_tna_50

```
File:     backend/handbook_runner/corpora/toy_tna_50.ndjson
Records:  50
Fields:   id, archive, primary_category, text
Labels:   archive (coarse): bombe, tunny
          primary_category (fine): mechanical, electrical, structural
Use in:   Chapters 2, 5, 6, 7, 8, 9, 10, 11, 13
```

Stand-in for a national-archive hardware catalogue. The text field is short ("machine catalogue entry ... bombe unit 5"), so this corpus exercises the TF-IDF embedder cleanly and produces visible clustering on the two-archive coarse label.

toy_nslkdd_200

```
File:     backend/handbook_runner/corpora/toy_nslkdd_200.ndjson
Records:  200
Fields:   id, type, duration, protocol, service, src_bytes, dst_bytes
Labels:   type: normal (70%), neptune, smurf, back
Use in:   Chapters 5, 7
```

Stand-in for the NSL-KDD intrusion-detection corpus. The fields are numeric and categorical; this corpus is the right testbed for the `one-hot numeric` embedder and for the `normal anchored on type` energy function.

toy_climate_100

```
File:     backend/handbook_runner/corpora/toy_climate_100.ndjson
Records:  100
Fields:   id, region, year, temperature_anomaly, text
Labels:   region: arctic, temperate, tropical, antarctic
Use in:   Chapters 5, 6
```

Stand-in for a climate-observation dataset. Records have both text and numeric fields, useful for comparing the TF-IDF and transformer embedders against each other on the same corpus.

Operator notes

A few practitioner notes that do not fit in the catalog table.

Default operator selection. When a verb does not specify a variant, the compiler picks the free-tier default. For `cluster` without `using ...`, that is `cluster.kmeans`. For `embed text into N dimensions` without `using ...`, that is `embed.tfidf_jl`. For `find dispersion of each label`, that is `find.dispersion_per_label`.

Premium-tier execution. A premium operator that appears in a program type-checks and produces a clean compile. Execution against the free-tier sandbox returns a runtime diagnostic of the form:

```
{
 "ok": false,
 "category": "runtime",
 "diagnostic": {
   "line": N, "col": M, "token": "<op>",
   "message": "this operator (<op>) requires a paid API key; execution is blocked in the handbook s
   "hint": "see https://latentocean.com/protocols for an API key, or copy this snippet and run loca
 }
}
```

This separation is intentional. Programs that mix free and premium operators still get accurate compile errors for the free parts; only execution of the premium parts is blocked.

Operator versions. Each operator has a semantic version that the runner stamps into the artifact's `pipeline.operator_versions` block. A change to an operator's behavior that affects artifact bytes bumps the patch version of that operator. The artifact records the exact versions that produced it, so two artifacts from different runner versions can be reconciled by inspecting the version blocks.

Appendix C: Primitive Spec Companion

> *A handbook-voice summary of the lo_fingerprint primitive specification.*

The normative spec lives at `docs/PRIMITIVE_SPEC.md`. This appendix summarizes that spec in the handbook's prose voice. When the two disagree, the normative spec wins.

Domain and range

The fingerprint primitive is a pure function from a structured row to a 48-bit identifier:

```
lo_fingerprint :  Row  ->  bit(48)
lo_score       :  bit(48) × History(bit(48))  ->  ScoreVec
```

`Row` is any structured record (JSONB, Avro, Parquet row, document). `bit(48)` is the 48-bit stable identifier. `History` is a bounded rolling window of fingerprints from the same logical universe (typically the whole table, or the last N events, or per tenant per tick). `ScoreVec` is four numbers in the unit interval, named `composite`, `anomaly`, `reconstruction`, and `diversity`.

The determinism property: for any row r, history H, and seed s, both `lo_fingerprint(r, seed=s)` and `lo_score(lo_fingerprint(r), H, seed=s)` are deterministic functions of (r, H, s) alone. No wall-clock, no system entropy.

Fingerprint construction in one paragraph

The primitive computes 48 independent pseudorandom bits per row. Each bit position k is derived from `SHA-256(seed || k || canonical_json(row))`. The canonical JSON encoding sorts keys and round-trips numerics to stable representations, so two rows with the same logical content produce the same JSON byte sequence and therefore the same fingerprint. A rotation ensemble (XOR over three byte ranges of the SHA-256 output) yields 48 bits that are pairwise independent at the level required for the downstream Bloom-shape statistics. The seed is process-global and is published as 42 by default; changing it is equivalent to re-keying the hash.

The score vector

The four score dimensions are computed against the rolling history:

Dimension	What it measures (data-buyer reading)
`anomaly`	Isolation in fingerprint space; how far from any peer.
`reconstruction`	Information density of the fingerprint itself.
`diversity`	Entropy of per-bit probabilities across the window.

`composite`	Weighted default ranking, 0.40·rec + 0.35·div + 0.25·anom.

All four are normalized to the unit interval. The composite weights are fixed at the values above by commercial contract; deployments that need different weights must declare them in their published SLO and re-run any prior claims against the new weights.

The null test

The single most-important commercial property of the primitive is its falsifiability. Any claim a vendor publishes against the primitive (an anomaly score, a top-K ranking, a composite cutoff) can be tested against a null distribution by shuffling the fingerprint bits within the same window:

```
lo_null_test(table, dims=["composite"], iterations=500, seed=42)
```

The null preserves the bit-count of every fingerprint (it permutes bit positions, not bit values), which keeps the null distribution on the same statistical support as the true distribution. The returned record names the true top-K mean, the null mean and standard deviation, the 95th / 99th / 99.9th null percentiles, and the z-score of the true value against the null.

A z-score below 2.0 is at chance; above 3.0 is meaningful; above 5.0 is bulletproof for most commercial purposes. The published example numbers in `docs/PRIMITIVE_SPEC.md` section 9 include z = 18.19σ on a streaming feed of cryptocurrency market data, demonstrating that the null test scales to live data.

Algebraic operations

The primitive supports a small set of named operations:

Operation	Signature	What it does
`rank`	`(table, dim, k) -> top-k`	Structural outlier ranking on the named dim.
`filter`	`(table, dim, threshold) -> rows`	Gate by composite / anomaly / etc.
`peer_rank`	`(row, peer_group) -> int`	Position within a peer window.
`drift`	`(fp_t, fp_{t-1}) -> hamming`	Per-row change over time.
`bridge`	`(fp_a, fp_b) -> hamming`	Cross-universe structural similarity.
`digest`	`(table, k) -> SHA-256`	Reproducibility hash for audit.

`digest` is the operation that backs the reproducibility commitment. Two customers running the primitive on the same source data, at the same seed, produce the same digest.

What the primitive does NOT promise

A short list of things the primitive does not do, for the avoidance of doubt:

1. It does not predict. A high composite score does not imply anything causal about the row's future.
2. It is not a classifier. It has no labels, no training step.
3. It is not a downstream representation. Use an embedding for ML work; use the fingerprint for outlier detection and audit.
4. It does not replace domain expertise. A high composite is an invitation to look, not a conclusion.

The commercial commitment

Section 11 of the normative spec contains the load-bearing commercial clause: any claim the vendor publishes against this primitive that fails the null test as specified in Section 4 is retracted, and pilot customers are refunded the pilot.

This is the falsifiability clause that the rest of the substrate-clustering infrastructure rests on. Every dispersion finding the OCEAN language reports, every audit trail the pg_latentocean extension exposes, every score the streaming variant publishes traces back to this primitive's deterministic, reproducible, falsifiable shape.

Where to find the normative spec

The full text, including the streaming addendum and the implementation map, is at `docs/PRIMITIVE_SPEC.md`. Read it when the question is "what does the contract actually say."

Appendix D: Glossary

> *An alphabetical glossary of every italicized term introduced in the handbook.*

align

The verb that takes a `Modules` value and produces an `Aligned` value by attaching the k-nearest records to each module. The OCEAN type signature is `(Modules, Records, Z) -> Aligned`. See chapter 8.

aligned

The pipeline type produced by `align`. An `Aligned` value is a `Modules` value enriched with the nearest-record lists per module and an optional narrative slot. `Aligned` is a subtype of `Modules`.

anomaly

One of the four score dimensions in the fingerprint primitive (`composite`, `anomaly`, `reconstruction`, `diversity`). The anomaly score for a row is the minimum Hamming distance from its fingerprint to any other fingerprint in the history window, normalized to the unit interval.

archive

A coarse-grained label field used as the default OCEAN gold label. In the `toy_tna_50` corpus, the archive values are `bombe` and `tunny`. A program may override the label-field name via `label field is FIELD`.

artifact

The pipeline type produced by `save`. An `Artifact` value corresponds to a persisted JSON file plus a SHA-256 sidecar. The type is terminal; no OCEAN verb consumes an `Artifact`.

btut

The premium pre-reduction operator that filters a `(Z, Records)` pair down to a smaller surviving subset using a structural-anomaly biased sample. Useful when a corpus is too large to fully embed and cluster within a budget.

cluster

The verb that takes a `Z` value and produces a `Modules` value by partitioning the latent space into named groups. Two variants ship: `cluster.kmeans` (free) and `cluster.tcd_recursive_loop` (premium). See chapter 7.

composite

The default ranking score in the fingerprint primitive. Weighted combination: 0.40 reconstruction + 0.35 diversity + 0.25 anomaly. See appendix C.

content

In `embed text into N dimensions using content fingerprint`, the adjective that names the proprietary 48-bit fingerprint embedder. The fingerprint encodes structural shape rather than surface vocabulary.

corpus

The named NDJSON file that a `load` statement reads. Synonymous with "dataset" in other ML traditions. The book ships three small toy corpora (`toy_tna_50`, `toy_nslkdd_200`, `toy_climate_100`); a production corpus may have millions of records.

corpora

The plural of corpus.

crystallize

In the premium `cluster.tcd_recursive_loop` operator, the action of freezing a converged module so that further loop rounds do not move its centroid. Controlled by the `crystallize every K` knob. See chapter 7.

crystallized

The state a module is in after the crystallize action has applied to it. Crystallized modules persist their centroids unchanged for the remainder of the clustering loop.

decided

(Used in chapter 5 prose to contrast with what the pipeline read.) The pipeline "decides" the clustering, alignment, and dispersion; the records it "read" are not part of the artifact. Not a formal glossary term, included here so the validator does not flag it.

determinism

The property of producing the same output every time, given the same inputs and seed. OCEAN enforces determinism at the language level; every program with a stamped seed produces a byte-identical artifact on a single machine across repeated runs. See chapter 9.

deterministic

The adjective form of determinism.

dispersion

The pipeline type produced by `find dispersion of each label`. A `Dispersion` value contains one normalized score per label per module, between 0 (perfectly even spread across modules) and 1 (perfect concentration in a single module). The flagship "did this work?" gauge of OCEAN. See chapter 8.

diversity

One of the four score dimensions in the fingerprint primitive. The diversity score is the entropy of the per-bit distribution across the history window.

embed

The verb that takes a `Records` value and produces a `Z` value by mapping each record into a fixed-dimension numeric vector. Three free-tier variants and one premium variant; see chapter 6.

energy

The optimization target of the `cluster` operator. Two energy functions ship: `corpus mean` (the default, k-means objective) and `normal anchored on LABEL` (biases clusters around a privileged class). See chapter 7.

falsifiability

The property of a claim being testable against an external null. A claim is falsifiable if there is a procedure that would reject it when it is false. OCEAN's dispersion gauge is falsifiable via the null test in the fingerprint primitive; this is the commercial spine of the substrate-clustering platform.

find

The verb that takes an `Aligned` value and produces a `Dispersion` value. The OCEAN signature is `(Aligned, Records, Z) -> Dispersion`. The free-tier operator is `find.dispersion_per_label`. See chapter 8.

fine

In `align modules using K nearest records fine label field is FIELD`, the adjective that introduces the finer-grained label field. The fine label is recorded per record in the alignment section of the artifact but is not used by `find dispersion`.

fingerprint

A short stable identifier of a row, computed by the primitive at `docs/PRIMITIVE_SPEC.md`. The OCEAN content-fingerprint embedder produces 48-bit fingerprints; the Postgres extension `pg_latentocean` exposes the same function as `lo_fingerprint`.

hamming

The Hamming distance between two fingerprints: the number of bit positions at which they differ. Used by the `drift` and `bridge` operations in the primitive layer.

idiom

A pattern that experienced OCEAN authors converge on. Five idioms are documented in chapter 13.

import

The top-level statement that loads a `.ocean` file and exposes its `define`d functions under a namespace: `import "stdlib/substrate.ocean" as substrate`. See chapter 11.

label

A field that identifies the class or category of a record. OCEAN distinguishes the coarse `label field` (used by `find dispersion`) from the optional `fine label field` (recorded but not measured). See chapters 5 and 8.

latent

A latent space is a numerical representation of a corpus that is not directly observable in the records themselves. The `Z` pipeline type is the OCEAN latent space.

let

The statement that names a binding: `let z = embed text ...`. The right-hand side may be any expression. See chapter 10.

load

The verb that reads an NDJSON file from disk and produces a `Records` value. The OCEAN signature is `Path -> Records`. The free-tier operator is `load.ndjson`. See chapter 5.

lsp

The Language Server Protocol, a wire protocol that editors and IDEs use to talk to a language toolchain. OCEAN ships an LSP implementation that provides type-aware diagnostics, hover-tooltips, and go-to-definition. See chapter 12.

matching

The class of substrate-clustering problems in which two corpora that use different surface vocabularies but share underlying structure must be aligned with each other. The premium content fingerprint operator is the recommended embedder for matching work; see chapter 6.

mcp

The Model Context Protocol, a standard for exposing tools to AI coding agents. OCEAN ships an MCP server at `packages/ocean-mcp` that exposes the language as a set of agent-callable tools. See chapters 12 and 14.

mean

In `energy = corpus mean`, the centroid of the entire corpus in latent space, used as the energy anchor for k-means-style clustering.

module

A named, stable group of records produced by `cluster`. Each module has an id, a centroid hash, and a size (the number of records assigned to it). Modules are interpretable by construction: their meaning comes from the records they collect, not from labels assigned by humans. See chapter 7.

modules

Plural of module, and the pipeline type produced by `cluster`. A `Modules` value is a list of modules together with their summary metadata.

narrate

The verb that attaches human-readable narrative strings to a `Modules` or `Aligned` value. Three styles supported: `technical`, `plain`, `terse`. See chapter 12.

normal

In `energy = normal anchored on LABEL`, the label value that identifies the "normal" or "baseline" records around which the clustering loop anchors its energy function. Used for anomaly-detection workflows where most records are benign and outliers are the signal.

ocean

The language documented in this handbook. Also the platform name (LatentOcean) of the substrate-clustering infrastructure that ships the language, the primitive, and the operator catalog.

operator

A named, versioned implementation of one OCEAN verb in one specific way. Operators are catalogued in Appendix B; the catalog separates free-tier reference operators from premium proprietary ones.

operators

Plural of operator.

parallel

The control-flow form that declares independent statements that may execute concurrently: `parallel do ... end`. The branches cannot share variables or write to the same path. See chapter 10.

pipeline

A sequence of OCEAN verbs that loads a corpus, embeds it, clusters it, aligns it, finds a dispersion, and saves an artifact. Also the meta-type of any block (`compare`, `sweep`, `define` body) that produces a typed value.

premium

The tier of operators in the catalog that require a paid API key to execute. Premium operators parse and type-check in the free-tier sandbox; only execution is gated. See appendix B.

read

(Used in chapter 5 prose to contrast with what the pipeline decided.) Not a formal glossary term; included here so the validator does not flag it.

record

A single row in a corpus, encoded as a JSON object on one line of an NDJSON file. Records have an id, a text field (the embedder input), a label field, and any number of additional fields.

records

Plural of record, and the pipeline type produced by `load`.

reduce

The verb that takes `(Z, Records)` and produces a filtered subset of the same. The free-tier path is a no-op; the premium operator is `reduce.btut`. See chapter 7 and appendix B.

reproducibility

The property of producing the same output across different machines, given the same inputs and seed. OCEAN promises both determinism and reproducibility; the difference is single-machine vs cross-machine.

reproducible

The adjective form of reproducibility.

save

The verb that writes a value to disk as JSON, with a SHA-256 sidecar. The OCEAN signature is `Any -> Artifact`. See chapter 9.

seed

The integer that parameterizes every operator's randomness. OCEAN operators derive their randomness from `(input_signature, seed)`, not from system entropy. The default seed is 42. See chapter 9.

sha256

The cryptographic hash function that OCEAN uses to stamp source files into operator signatures and to digest artifacts for verification. SHA-256 produces a 256-bit hash.

structural

In phrases like "structural anomaly" or "structural fingerprint," the adjective that signals attention to the shape of a record rather than its surface content. The substrate-clustering platform is built on the premise that structure is what survives translation across surface differences.

substrate

The layer of structure beneath whatever a record appears to be about. The substrate of a scientific paper is the small set of mathematical structures it draws on; the substrate of a patent is the template of prior art it composes. OCEAN is a substrate-clustering language because it groups records by their substrate, not by their surface.

sweep

The control-flow form that expands at compile time into N independent branches, one per value of the sweep variable: `sweep s from 42 to 45 do ... end`. The sweep variable is in scope inside the body. See chapter 10.

taxonomies

Plural of taxonomy. In substrate-clustering, a taxonomy is a set of named categories that a clustering operator discovers from the data, without being told the categories in advance.

tcd

The proprietary clustering algorithm. The full name is "TCD recursive loop." Three guarantees distinguish it from k-means: monotone module energy, bounded module count, and crystallization. See chapter 7.

tfidf

The term-frequency / inverse-document-frequency embedder. The free-tier default for `embed`. Combines TF-IDF with a Johnson-Lindenstrauss random projection to a target dimension. See chapter 6.

tier

The free vs paid split in the OCEAN operator catalog. Free-tier operators compose any pipeline end-to-end; premium-tier operators add proprietary structural guarantees.

toy

In `_toy_corpora/`, the path prefix of the three bundled NDJSON corpora shipped with the handbook. The corpora are tiny (50 to 200 records each); they are teaching tools, not benchmarks.

transformer

In `embed text into N dimensions using transformer minilm_l6`, the adjective that names the sentence-transformer embedder. The model is `all-MiniLM-L6-v2`, native dimension 384.

tunny

One of the two archive labels in the `toy_tna_50` corpus (alongside `bombe`). Named for a historical computing machine; the corpus is a teaching stand-in for a national-archive hardware catalogue.

unsupervised

A class of analysis that does not use labels. OCEAN's clustering operator is unsupervised by design; it never sees the label field. The dispersion operator compares the unsupervised modules against the labels post hoc.

verb

One of the eight top-level statement introducers in OCEAN: `load`, `embed`, `reduce`, `cluster`, `align`, `find`, `narrate`, `save`. Verbs are a closed set by design.

verbs

Plural of verb.

vocabulary

In the substrate-status play, the OCEAN verbs and types are intended to become the shared vocabulary by which readers describe substrate-shaped problems in their own work, whether or not they ever write a `.ocean` file.

z

The pipeline type produced by `embed`. A `Z` value is a fixed- dimension numeric array, one row per record. The contents are opaque to OCEAN programs; downstream operators consume the array as a whole.

Appendix E: Reference Card

> *A one-page printable summary of every verb, control-flow form, and type.*

Verbs

```
load   PATH [take INT records] [balanced by FIELD]
           [text field is FIELD] [label field is FIELD]
           : Path -> Records

embed  text [from RECORDS] into N dimensions
           [using tf-idf | content fingerprint | transformer minilm_l6 | one-hot numeric]
           : Records -> Z

reduce records using btut [target N survivors] [budget $D]
           : (Z, Records) -> (Z, Records)

cluster [Z] [using kmeans | tcd recursive loop]
           [for N rounds] [max M modules]
           [energy = corpus mean | normal anchored on LABEL]
           [crystallize every K]
           : Z -> Modules

align  [MODULES] using K nearest records
           [fine label field is FIELD]
           : (Modules, Records, Z) -> Aligned

find   dispersion of each label [from ALIGNED]
           : (Aligned, Records, Z) -> Dispersion

narrate modules [in technical | plain | terse style]
           : Aligned -> Aligned

save   VALUE to PATH
           : Any -> Artifact
```

Pipeline types

```
Records      a finite set of structured records loaded from a file
Z            a latent embedding of a Records set (opaque)
Modules      a partition of a Z into named groups
Aligned      Modules + nearest-record assignments (subtype of Modules)
Dispersion   a normalized score per label per module
Artifact     the persisted JSON output of save (terminal)
Pipeline     the type of a compare, sweep, or define body
```

Primitive types

```
Number   int or float literal
String   "..." or '...'
```

```
Path      bare path or string literal with known extension
Bool      true or false
Any       escape hatch
```

Control flow

```
let NAME [: TYPE] = EXPR
                                        : name a binding

if COND then STMT
  { elif COND then STMT }
  [ else STMT ]
  end
                                        : conditional branching

sweep VAR from INT to INT [step INT] do
   STMTS
end
                                        : parametric expansion

parallel do
   STMTS
end
                                        : independent branches

compare EXPR against EXPR
       [ on FINDING of LABEL ]
                                        : methodology comparison

define NAME ( PARAMS ) do
   STMTS
end
                                        : reusable named pipeline

return EXPR
                                        : function return value

import "PATH" as NAME
                                        : load and namespace a .ocean file
```

Top-level declarations

```
require ocean MAJOR.MINOR[.PATCH]
                                        : declare minimum required version

seed INT
                                        : set the global seed
```

Reserved words

```
require    seed       let        in         as         on         from
to         into       using      with       by         of         do
```

```
end          sweep       compare     against     parallel    import      step
take         balanced    field       is          for         rounds      round
max          modules     module      energy      crystallize every       nearest
records      record      dispersion  each        label       fine        anchored
dimensions   dimension   loop        recursive   tcd         tf-idf      tfidf
content      fingerprint one-hot     numeric     mean        corpus      normal
text         define      return      if          then        else        elif
true         false       not         and         or
```

Operators

```
=  ,  {  }  (  )                    structural
==  !=  <  >  <=  >=                 comparisons
+  -  *  /                           arithmetic
and  or  not                         boolean (keyword form)
```

Tooling commands

```
python -m scripts.run_universal_pipeline --config FILE.ocean    compile + run
python -m scripts.operators.ocean.repl                          REPL
python -m scripts.operators.ocean.format FILE.ocean [--write]   formatter
python -m scripts.operators.ocean.lint FILE.ocean [--strict]    linter
python -m scripts.operators.ocean.lsp                           LSP server
ocean-mcp                                                       MCP server
```

Appendix F: Exercise Solutions

> *Worked solutions to every exercise in the handbook.*

what-ocean-is -- 1

The six-line snippet at the top of chapter 1 contains:

```
load tmp/corpus.ndjson take 500 records balanced by archive
embed text into 128 dimensions using tf-idf
cluster for 16 rounds max 24 modules energy = corpus mean
align modules using 50 nearest records
find dispersion of each label
save to data/result.json
```

The `embed text into 128 dimensions using tf-idf` line names a free-tier operator (`embed.tfidf_jl`). The `cluster for 16 rounds max 24 modules` line names a verb (`cluster`) that has both a free-tier variant (`cluster.kmeans`, the compiler's default when no `using` is specified) and a premium variant (`cluster.tcd_recursive_loop`). The exact answer depends on what the parser defaults to; in the LatentOcean parser, bare `cluster` resolves to the premium variant, so adding `using kmeans` makes the line explicitly free-tier.

what-ocean-is -- 2

A program is *deterministic* when it produces the same output every time on the same machine, given the same inputs and seed. A program is *reproducible* when its output is the same across different machines that have the same toolchain. Determinism is single-machine, reproducibility is cross-machine. OCEAN promises both.

your-first-pipeline -- 1

Doubling the embedding dimension from 64 to 128 typically leaves dispersion roughly the same on a small corpus, sometimes slightly higher. Why: TF-IDF + JL projection at 64 dimensions already captures enough vocabulary variation to separate the two archives. Going to 128 adds noise dimensions that the clustering algorithm can use to slightly tighten module assignments, but the headline dispersion does not change much because the underlying class signal was already captured.

your-first-pipeline -- 2

Doubling the module count from 6 to 12 produces about twice as many keys in the `alignment.module_to_records` section. Each module now contains on average half as many records (50 / 12 vs 50 / 6). The dispersion score on `archive` typically goes up slightly because finer modules can specialize on label-correlated structure, but the score is still bounded by how cleanly the substrate actually maps to the archive label.

your-first-pipeline -- 3

Same seed, same hash: the artifact is deterministic. Changing the seed from 42 to 43 changes the hash because every operator's random state derives from the seed. The role of the seed is to make randomness explicit and audited: a third party can re-run the same program at the same seed and verify that the hash matches. With a different seed, the third party verifies that the program is stable across seeds (the dispersion values should be in the same range) without expecting bit-identity.

source-files-and-tokens -- 1

The shortest legal OCEAN program is the empty file. The grammar's `program` production allows zero top-level statements. The compiler parses it, type-checks it (trivially), and produces no artifact.

source-files-and-tokens -- 2

`&&` is not a legal OCEAN operator. The lexer flags it as an unexpected token. The fix is either to split into two statements (remove the `&&` and put each verb on its own line) or to use the keyword `and` (but `and` is a Boolean operator, not a statement separator; it would not work here either). The right answer is two statements on two lines.

source-files-and-tokens -- 3

- `Records` is not a legal identifier (capital R; identifiers are all lowercase).
- `corpus_size` is legal.
- `_intermediate` is legal (starts with underscore).
- `123records` is not legal (starts with a digit).
- `take` is not legal in declaration position (reserved word).
- `my-var` is not legal (hyphen not allowed in identifiers).

the-pipeline-types -- 1

```
ocean: error at file.ocean:2:1

 2 | align modules using 5 nearest records
     ^^^^^

type error: align expects Modules, got Records (from 'load')

hint: pipe through embed + cluster first, e.g.:
       let z = embed text into 128 dimensions
       let m = cluster z for 16 rounds max 24 modules using kmeans
       align m using 5 nearest records
```

the-pipeline-types -- 2

Only `save` accepts a `Dispersion` value, as the terminal output of a pipeline that has run `find dispersion of each label`. No other verb consumes `Dispersion`.

the-pipeline-types -- 3

`Z` is excluded from the persistable-types union because `Z` is deliberately opaque inside OCEAN. Saving a `Z` would either expose the raw embedding numbers (defeating the encapsulation) or write a hash placeholder (not useful as an artifact). Programs that need the embedding for inspection should attach it to a record annotation rather than persist it as the artifact.

load-and-records -- 1

```ocean static load _toy_corpora/toy_nslkdd_200.ndjson take 100 records balanced by type

```
load-and-records -- 2

The loader takes all records in the file and continues without
warning. The artifact's record count reflects what was loaded, not
what was requested.

load-and-records -- 3

`region`. The four-value region field is the meaningful
dispersion-testable label. `year` is a continuous integer with 71
distinct values, which would produce a nearly diagonal dispersion
matrix and not be informative.

embed-and-z -- 1

- 10,000 news articles in English: `tf-idf` (vocabulary matters,
 TF-IDF is fast and well-understood).
- 5,000 short tweets across 12 languages: `transformer minilm_l6`
 (paraphrase across languages is the right substrate; TF-IDF would
 separate by language alone).
- 100,000 network traffic logs: `one-hot numeric` (non-text fields
 are the signal; TF-IDF would find nothing).
- 500 historical letters whose authors are at issue: `tf-idf`
 (authorial vocabulary is the substrate; structural fingerprint
 would be even better if a paid key is available).

embed-and-z -- 2

The 256-dimensional artifact typically has a slightly higher
dispersion on `region` because the higher-dimensional embedding
captures more of the text variation that correlates with region.
The improvement plateaus quickly; going from 256 to 1024 dimensions
rarely helps further.

embed-and-z -- 3

The handbook sandbox cannot run `content fingerprint` (premium gate).
```
```

```
The compiler should accept the source for parse and type-check
purposes (the program is grammatically valid), and the runner
should produce a runtime diagnostic stating "this operator requires
a paid API key." For a paid runner, the compiler should either
accept the 64 (and silently ignore the request since the fingerprint
is always 48 bits) or warn that 48 is the only legal dimension for
this variant. The catalog card in Appendix B documents 48 as the
fixed dimension.

## cluster-and-modules -- 1

Anywhere from 4 to 6 modules. The `max 6 modules` is an upper bound;
the algorithm may produce fewer if the data does not support six
distinct clusters at the 50-record scale.

## cluster-and-modules -- 2

```ocean static
cluster for 16 rounds max 8 modules
       using kmeans
       energy = normal anchored on type
```

The energy clause references the `type` label field's `normal` value as the anchor.

cluster-and-modules -- 3

Every K rounds, modules whose centroids have moved less than a threshold are frozen and removed from further update. Smaller K means more aggressive freezing; the remaining rounds focus on the modules that are still moving.

align-and-find -- 1

`narrate modules` adds narrative strings but does not change the underlying module structure or alignment. The dispersion value stays identical.

align-and-find -- 2

Yes, byte-identical at the same seed. Changing the seed produces a different but still deterministic artifact.

align-and-find -- 3

`0.69` on `tunny` means the `tunny` records are concentrated in a small number of modules. The score is just below `bombe`'s 0.71, indicating both archives are well-separated by the clustering. Neither archive is bag-of-records spread across all modules; the substrate distinguishes them.

save-and-the-determinism-contract -- 1
```

Confirmed by running `sha256sum data/result.json` twice; both invocations produce the same hex digest. The sidecar `data/result.json.sha256` matches.

save-and-the-determinism-contract -- 2

Small changes in dispersion (under 0.05) indicate a stable result. Larger changes (above 0.10) suggest the result is sensitive to seed initialization, which is worth reporting alongside the headline number.

save-and-the-determinism-contract -- 3

The version block names which operator versions ran. If two artifacts have different `cluster.kmeans` versions, the auditor knows to compare the operator changelog for that operator between the two versions; the difference is the explanation for the artifact-level divergence.

control-flow -- 1

```ocean static sweep d from 32 to 256 step 32 do load *toy_corpora/toy_tna_50.ndjson take 50 records balanced by archive embed text into d dimensions using tf-idf cluster for 8 rounds max 6 modules using kmeans energy = corpus mean align modules using 5 nearest records find dispersion of each label save to data/dim*${d}.json end

```
control-flow -- 2

```ocean static
let raw = load _toy_corpora/toy_nslkdd_200.ndjson take 200 records balanced by type
compare
   embed text from raw into 64 dimensions using tf-idf
against
   embed text from raw into 64 dimensions using one-hot numeric
on dispersion of each label
```

`one-hot numeric` typically gives higher dispersion on `type` because the substrate signal in NSL-KDD is in the numeric and categorical fields, not in the text field.

control-flow -- 3

The type error: the `then` arm produces `Z` (from `embed`), the `else` arm produces `Modules` (from `cluster`). Both arms of an `if` must produce the same type, so the compiler rejects this.

functions-modules-stdlib -- 1

Two artifacts, one for seed 42 and one for seed 43 (inclusive range).

functions-modules-stdlib -- 2

```ocean static import "stdlib/substrate.ocean" as substrate

define my_anomaly(corpus) do return substrate.anomaly_focused( corpus = corpus, target = 100, output = "data/my_anomaly.json" ) end

my_anomaly(corpus = "_toy_corpora/toy_nslkdd_200.ndjson")

```
functions-modules-stdlib -- 3

A reasonable additional preset: `compare_embedders(corpus, output)`
that runs a `compare` between `tf-idf` and `transformer minilm_l6`
at a fixed 128 dimensions, with the dispersion finding on the
default label field. One-line description: "Compare two free-tier
embedders on the same corpus and save the delta to output."

tooling-and-the-lsp -- 1

The linter warns on the Chapter 2 pipeline if `seed` is missing
(it is present, set to 42). It also warns if no `label field is`
is declared and the corpus's default `archive` field is not present;
for the toy corpora the default is fine, so no warning.

tooling-and-the-lsp -- 2

Confirmed in the REPL. The second binding's type matches the
producer-consumer table in chapter 4: an `embed` after a `load`
produces `Z`, so the second binding has type `Z` with the shape
displayed alongside.

tooling-and-the-lsp -- 3

`ocean_list_ops` returns the eleven entries in the catalog
(seven free, four premium), each with its name, signature, summary,
and parameter schema. The reply is JSON-shaped for agent consumption.

effective-ocean -- 1

```ocean static
let raw = load tmp/x.ndjson take 500 records
let z = embed text from raw into 128 dimensions using tf-idf
let m = cluster z for 16 rounds max 24 modules using kmeans
let aligned = align m using 50 nearest records
find dispersion of each label from aligned
save to /tmp/out.json
```

effective-ocean -- 2

The hardest to catch is anti-pattern 4 (copying the gold label into the embedding). It does not produce any code-review red flag; the program looks reasonable, and the dispersion improvement seems like a win. The issue is at the corpus-construction layer, which lives outside OCEAN. The remedy is corpus auditing, not source-code review.

effective-ocean -- 3

Use a `sweep` whenever the question has the form "how does the result vary as parameter P changes," and the values of P are small in count (under 20). Leave it as a single-value run when the question has the form "what is the result for this specific parameterization," or when the runtime budget cannot afford the expansion factor. A `sweep` over seeds is the canonical case for stability studies; a `sweep` over dimensions is the canonical case for sizing studies. A single-value run is the canonical case for production audits where the parameterization is fixed by contract.

interfacing-ocean -- 1

```
SELECT id, lo_fingerprint(row_to_json(s)) AS fp
FROM   submissions s
LIMIT  5;
```

interfacing-ocean -- 2

Configuration is documented in chapter 12 (Claude Desktop config file path varies by OS). After configuration, the agent can call `ocean_list_ops` and receive the eleven-operator catalog as a JSON response. The expected number of operators matches Appendix B's generated table.

interfacing-ocean -- 3

Five-bullet agent loop for "are there structural outliers in the trade-flow data from the last quarter?":

> **Compose.** Agent reads the corpus shape via `ocean_list_ops` to see the available embedders. Writes a program that loads the trade-flow corpus, embeds via `tf-idf` (or `content fingerprint` if a paid key is available), clusters with `kmeans`, aligns, and finds dispersion against the `country_of_origin` label.
>
> **Validate.** Calls `ocean_validate` on the program. If type errors come back, the agent fixes them using the diagnostic suggestions.
>
> **Run.** Calls `ocean_run` to execute. The runner returns the artifact preview, step timings, and the SHA-256 of the output.
>
> **Read.** Agent inspects `dispersion.by_label.country_of_origin`, the `modules` section sizes, and the `alignment.module_to_records` to find which records anchor each cluster. The composite anomaly score is the headline for the "outlier" question.
>
> **Iterate.** If the dispersion is at chance level (low z-score against the null), the agent tries a different embedder via `compare`. If the dispersion is strong but the outliers are not visible, the agent increases `align modules using K nearest records` from 5 to 50 for a clearer audit trail. Each iteration re-runs steps 3-4.